NGUYEN DEVELOPMENT · PHPDBADMIN

CR-001 — phpdbadmin

Change Request mit Ideensammlung, Rückfragen, Marktbild, Funktionsraster und Ideen-Backlog

Produkt	phpdbadmin — Datenbank-Verwaltung im Browser für MariaDB, MySQL, PostgreSQL und SQLite
Träger	Nguyen Development (Einzelfirma), Hoang Nguyen · Lizenz MIT
Vorgehen	DSDM/AgilePM — Dokumentation in dokumentationen/, ein PDF je Phase
Stand	13.11.2026, 17:41 · baut auf Commit 76958de + Änderungen dieses Standes auf
Quelle	dokumentationen/ im Repo — dieses PDF wird bei jeder Änderung neu gebaut

Dieses Dossier enthält

1. CR-001 — phpdbadmin: Datenbank-Verwaltung im Browser für MariaDB, MySQL, PostgreSQL und SQLite
2. Anhang A — Ideensammlung — phpdbadmin
3. Anhang B — Ausgangslage und Rückfragen — phpdbadmin
4. Anhang C — Marktbild — Datenbank-Werkzeuge 2026
5. Anhang D — Funktionsraster — phpMyAdmin, phpPgAdmin, Adminer → phpdbadmin
6. Anhang E — Ideen-Backlog — phpdbadmin

Inhalt

CR-001 — phpdbadmin: Datenbank-Verwaltung im Browser für MariaDB, MySQL, PostgreSQL und SQLite

1. Ausgangslage und Problem
2. Ziel und Positionierung
3. Scope
4. Anforderungen
5. Lösungsansatz und technischer Rahmen
6. Geschäftsmodell
7. Erwarteter Nutzen
8. Alternativen
9. Aufwand und Stufen
10. Risiken auf oberster Ebene
11. Rahmen und Abgrenzung
12. Abhängigkeiten und Voraussetzungen
13. Offene Punkte
14. Entscheid und nächste Schritte

Anhang A — Ideensammlung — phpdbadmin

1. Ausgangslage
2. Warum — Problem und Lücke
3. Für wen — Zielgruppen und Rollen
4. Was — Funktionsumfang
5. Was nicht — Abgrenzung (Vorschlag)
6. Wie — Produktprinzipien (Vorschlag)
7. Womit — Technik (Vorschlag)
8. Wo — Betrieb und Umgebungen
9. Wieviel — Geschäftsmodell (offen)
10. Wann — Fahrplan (Grobvorschlag)
11. Wer — Rollen
12. Woran gemessen — Erfolgskriterien (Vorschlag)
13. Offene Entscheide

Anhang B — Ausgangslage und Rückfragen — phpdbadmin

1. Was feststeht
2. Was ich selbst nachgelesen habe (keine Rückfrage nötig)
3. Rückfragen
4. Entscheide aus der interaktiven Runde vom 28.10.2026
5. Entscheide aus dem Review der Foundation vom 28.10.2026

Anhang C — Marktbild — Datenbank-Werkzeuge 2026

1. Übersicht
2. Was der Markt zeigt
3. Positionierung
4. Geschäftsmodelle im Markt
5. Risiken aus dem Markt
6. Quellen (abgerufen 28.10.2026)

Anhang D — Funktionsraster — phpMyAdmin, phpPgAdmin, Adminer → phpdbadmin

1. Anmeldung und Zugang
2. Verbindungen
3. Bedienung
4. Daten
5. Struktur
6. SQL
7. Export und Import
8. Server-Verwaltung
9. Zusammenfassung
10. Feinabgleich: Operationen für Datenbank und Tabelle (13.11.2026)

Anhang E — Ideen-Backlog — phpdbadmin

1. Anmeldung und Konten
2. Verbindungen
3. Bedienung
4. Daten
5. Struktur und SQL
6. Export, Import, Betrieb

Architektur-Hinweis

CR-001 — phpdbadmin: Datenbank-Verwaltung im Browser für MariaDB, MySQL, PostgreSQL und SQLite

Feld	Wert
Nummer	CR-001
Titel	phpdbadmin — ein Werkzeug für vier Datenbanken, intuitiver als phpMyAdmin und phpPgAdmin
Antragsteller	Hoang Nguyen
Datum	28.10.2026
Fassung	6
Status	✅ **Weiterführen** entschieden 28.10.2026 (Hoang: «Feasibility starten») · Feasibility abgeschlossen 28.10.2026: 6 bestanden, 4 bedingt — Auflagen als V-10, Q-09, E-06 und geänderte Abnahmen eingearbeitet
Grundlagen	Ideensammlung · Marktbild · Funktionsraster · Ausgangslage und Rückfragen · Ideen-Backlog
Folgt	Terms of Reference (Entwurf v0.1) → Feasibility

Änderungsverlauf

Fassung	Datum	Änderung
1	28.10.2026	Erste Fassung aus Hoangs Auftrag, Recherche und Funktionsraster; Anforderungen A, V, O, D, S, Q, E, W, B, N mit Abnahme
2	28.10.2026	Interaktive Rückfragen: R-01 und R-40 (Nguyen Development, MIT), R-11 (lokal und selbst gehostet), R-03 (neben Basalto), R-04 (Name phpdbadmin endgültig) — § 6, § 10, § 11, § 14 nachgeführt
3	28.10.2026	Entscheid Weiterführen; Ergebnisse der Feasibility: neu V-10, Q-09, E-06; Abnahmen von V-02, V-04, O-01, D-04, S-02, S-03, Q-02, Q-03, E-01, B-04, N-02, N-03 geschärft; Schätzung 42–53 PT
4	28.10.2026	Review der Foundation (Hoang): neu **V-11** Schreibschutz auf Zeit aufheben; D-04 zählt die Folgen über `ON DELETE CASCADE` in der Vorschau (aus den Wireframes)
5	28.10.2026	TB 00, Wunsch Hoang: **A-06** — erstes Konto auch im Browser, abgesichert über einen Einrichtungsschlüssel als Datei auf dem Server (wer kein Terminal hat, kommt über Dateimanager oder FTP an die Datei), danach Passkey-Onboarding
6	13.11.2026	Wunsch Hoang («ich vermisse etliche Datenbankfunktionen … db dump, backup, optimierung, repair»): Feinabgleich mit phpMyAdmin im Funktionsraster § 10; neu **O-10** (Export, Import, Sicherung und Wartung aus den Menüs von Datenbank und Tabelle), **W-06** (Reparieren, Prüfsumme, Indizes neu aufbauen), **S-10** (Tabellenoptionen), **S-11** (Tabelle in eine andere Datenbank kopieren oder verschieben), **S-12** (Zeichensatz einer Datenbank)

1. Ausgangslage und Problem

1.1 Woher die Idee kommt

Hoang arbeitet täglich mit vier Datenbanken: MariaDB und PostgreSQL lokal in Herd, SQLite in Tests und kleinen Installationen, MySQL bei Kunden und in den Pipelines von Basalto. Die Werkzeuge dafür sind getrennt, alt oder nicht im Browser. Der Auftrag vom 28.10.2026: ein Admin-Werkzeug wie phpMyAdmin

oder phpPgAdmin, aber intuitiver, benutzerfreundlicher und für alle vier Plattformen — auf Laravel, ohne Passwort, ohne Dashboard und ohne Profilseite des Starter-Kits.

1.2 Das Problem

phpdbadmin — Problembild: heute und mit phpdbadmin

Wer MariaDB, MySQL, PostgreSQL und SQLite im Browser verwalten will, braucht heute mehrere Werkzeuge — keines führt durch die Arbeit

HEUTE

phpMyAdmin 5.2.3
nur MySQL und MariaDB
offiziell nur PHP < 8.4
Version 6 seit Jahren
in Entwicklung
viele Reiter und Formulare
Anmeldung mit dem
Datenbank-Passwort

phpPgAdmin 7.13
nur PostgreSQL
seit Dezember 2023
ohne Commit
offene XSS-Meldung 2026
neuere PostgreSQL-
Versionen nur in Forks
Oberfläche der 2000er

Adminer 6.2
alle vier, eine PHP-Datei
aktiv gepflegt
knapp und schnell
Bedienung über Links
und Formulare
wenig Führung, keine
Vorschau vor dem Ändern

Desktop-Clients
TablePlus, DBeaver,
Beekeeper, HeidiSQL
je Rechner installiert
nicht im Browser,
nicht auf dem Server
teils kostenpflichtig
oder mit Tab-Grenzen

Gemeinsam: keine einheitliche Bedienung über vier Datenbanken · Änderungen ohne Vorschau des SQL · kein Schutz vor DROP auf der Produktion · Anmeldung mit Datenbank-Passwörtern statt mit eigenem Konto

MIT PHPDBADMIN

phpdbadmin
eine Oberfläche für alle vier Datenbanken
Tabelle bearbeiten wie in einer Tabellenkalkulation
SQL-Vorschau vor jedem Schreiben
Befehlspalette ⌘K für alles
Produktion farbig und schreibgeschützt
Anmeldung mit Passkey oder Code
Zugangsdaten verschlüsselt gespeichert
Laravel 13, Inertia v3, Vue 3, Tailwind 4
läuft, wo Laravel läuft (Herd, Server)

Was sich ändert

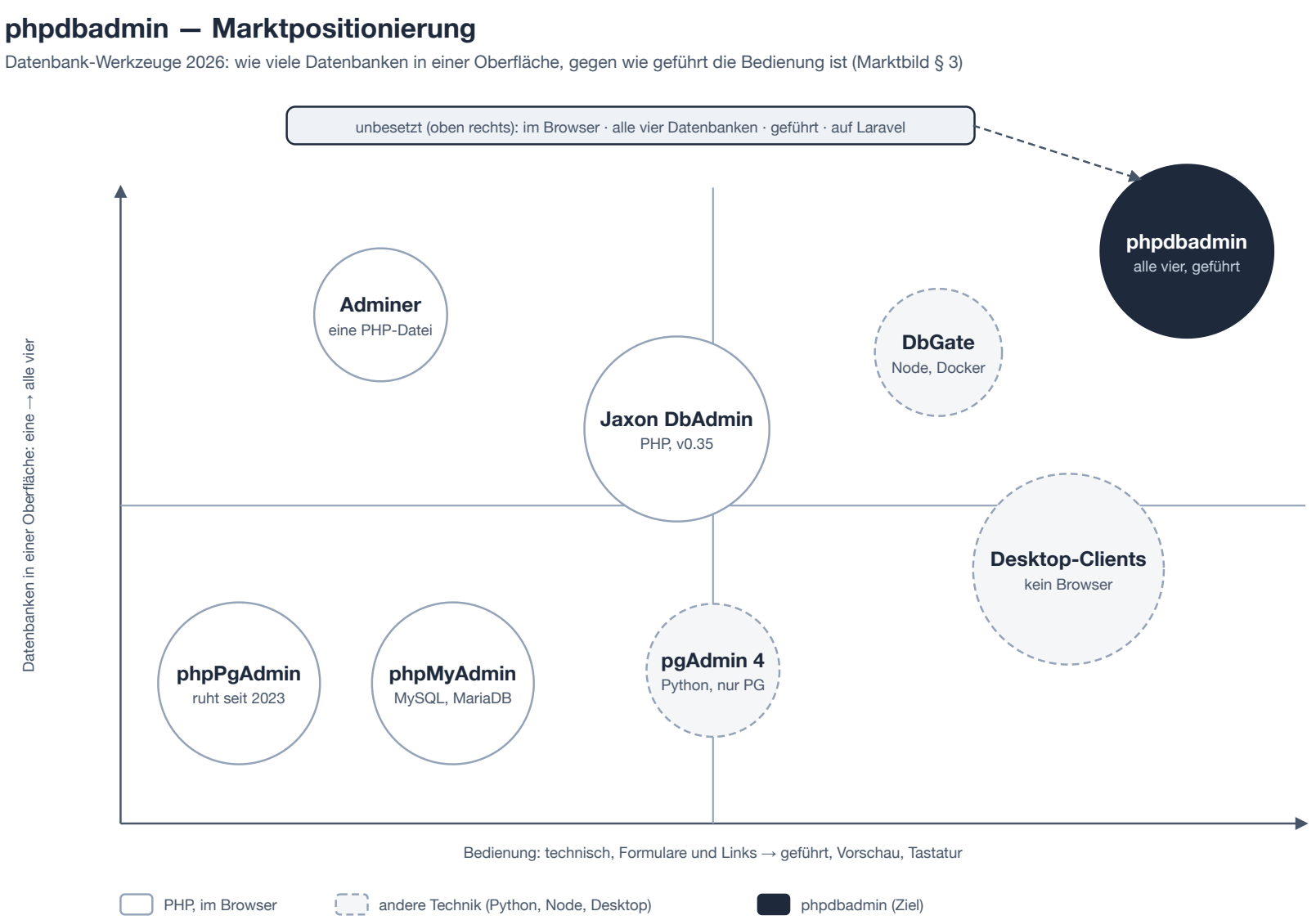
Entwickler	Lokale Datenbanken aus Herd in einer Oberfläche, egal ob MariaDB oder PostgreSQL; SQL lernen am gezeigten SQL
Betrieb / Admin	Ein Werkzeug auf dem Server statt phpMyAdmin und phpPgAdmin; Konten mit Passkey, Protokoll, Schreibschutz
Nguyen Development	Eigenes Werkzeug für alle Projekte (Basalto, Fristo, Kunden); Schaufenster für Laravel-Handwerk

Ein Werkzeug je Datenbank, Bedienung über Formulare und Links, Änderungen ohne Vorschau und ohne Rückweg, Anmeldung mit Datenbank-Passwörtern, und die PHP-Klassiker stehen technisch still (Ideensammlung § 2).

1.3 Marktlage

phpMyAdmin 5.2.3 ist offiziell nicht für PHP 8.4 und neuer freigegeben, phpPgAdmin ruht seit 2023, Adminer ist breit, aber knapp; die modernen Oberflächen (DbGate, CloudBeaver, Desktop-Clients) sind nicht PHP. Jaxon DbAdmin zielt seit Oktober 2026 in dieselbe Lücke und steht bei Version 0.35 (Marktbild § 2).

2. Ziel und Positionierung

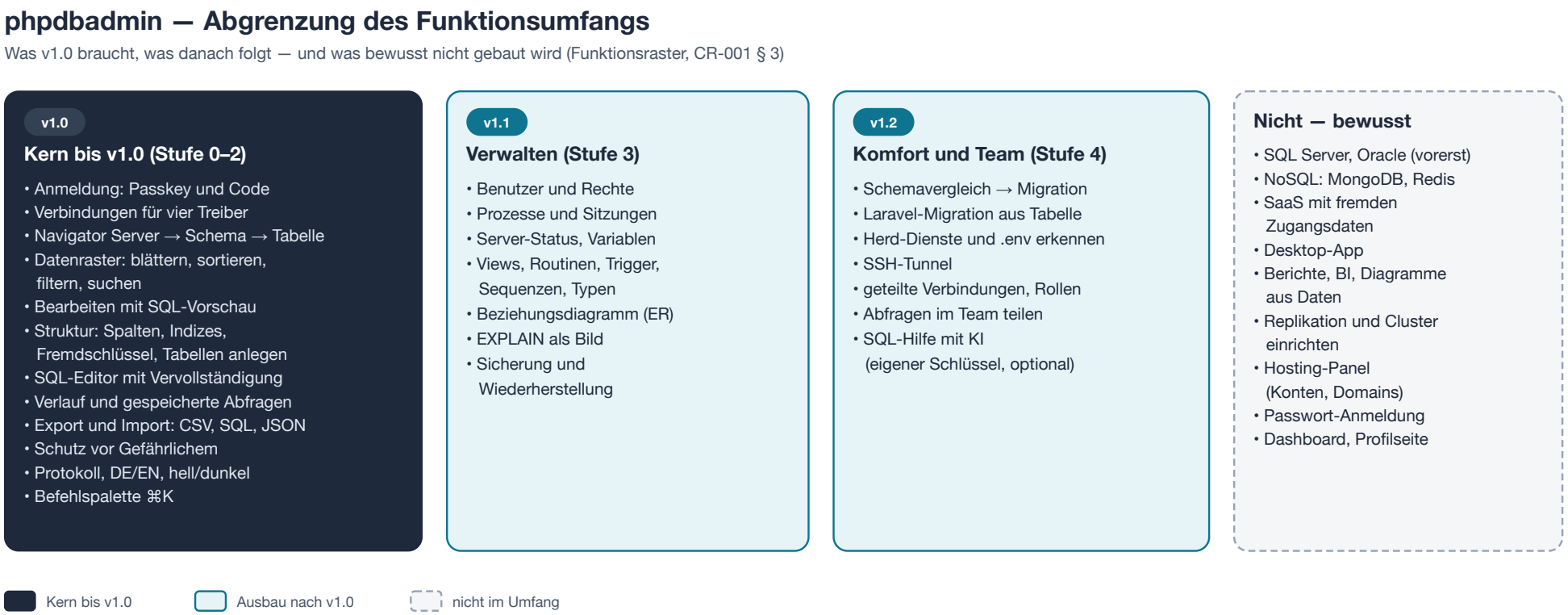


Ziel: Ein Werkzeug im Browser, mit dem man MariaDB, MySQL, PostgreSQL und SQLite mit **derselben Bedienung** ansieht, bearbeitet, strukturiert und austauscht — **geführt**, **sicher** und **schnell**, lokal in Herd wie auf einem Server.

Positionierung (Vorschlag, Marktbild § 3): «Ein Werkzeug für alle vier, das führt» mit «Sicher im Team» im Kern und «Für Laravel-Entwickler» als Unterscheidungsmerkmal nach v1.0. Leitsatz-Entwurf: *Alle vier Datenbanken. Eine Oberfläche. Nichts passiert ungesehen.*

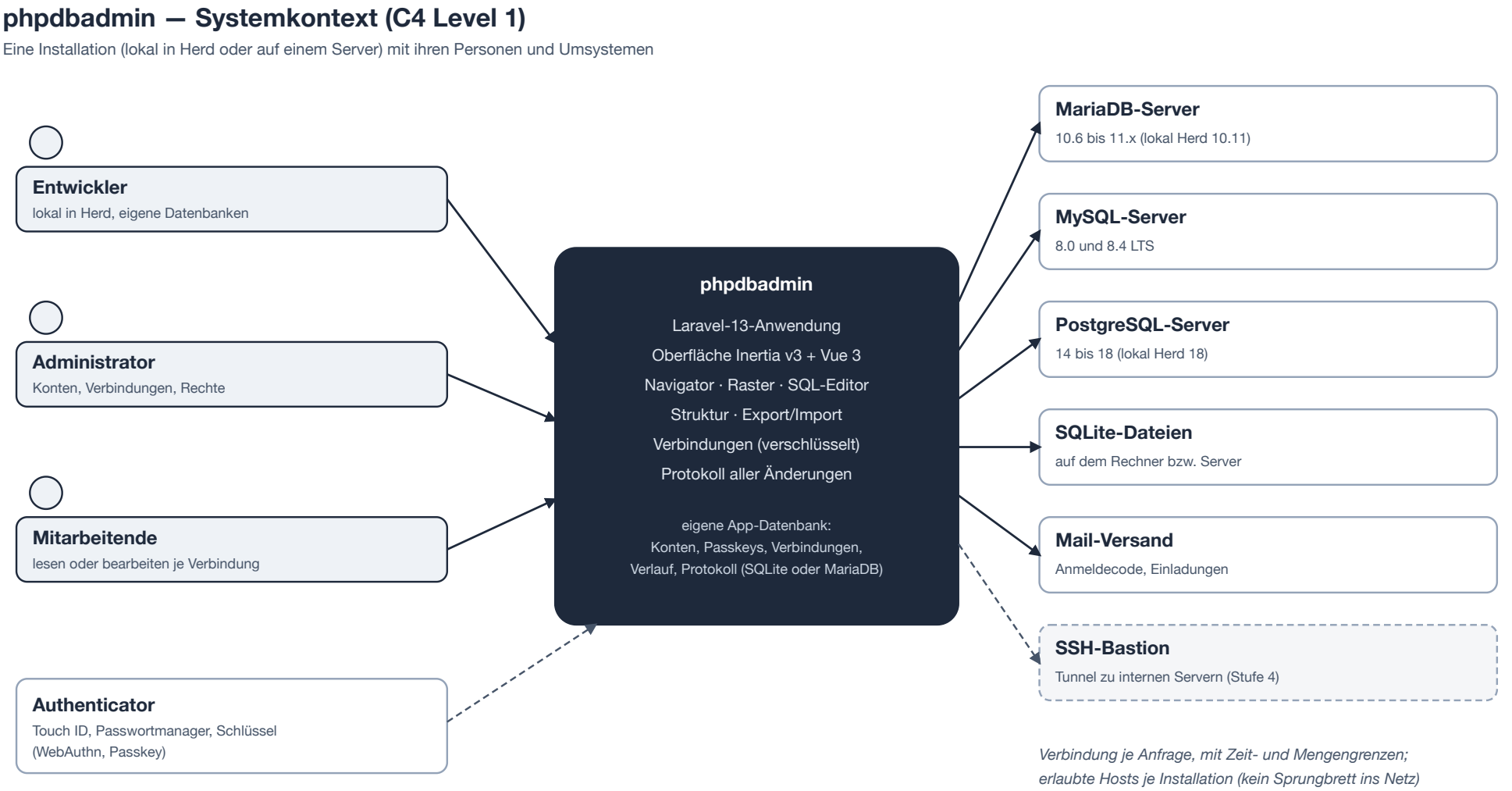
3. Scope

3.1 Abgrenzung



v1.0 umfasst Anmelden, Verbinden, Lesen, Bearbeiten und Austauschen (Stufen 0–2); v1.1 die Server-Verwaltung (Stufe 3); v1.2 Komfort und Team (Stufe 4). Die Einordnung jeder einzelnen Funktion steht im Funktionsraster.

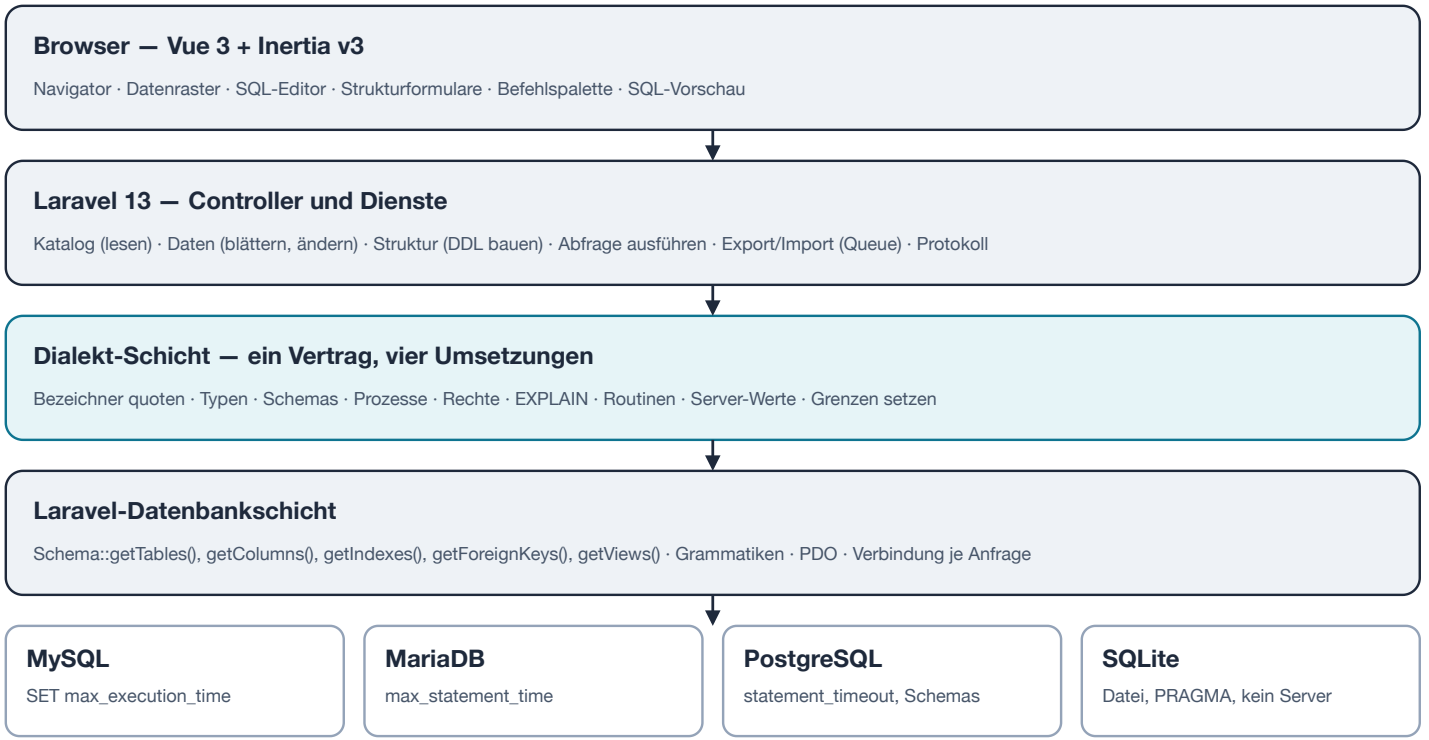
3.2 Architektur



phpdbadmin ist eine Laravel-13-Anwendung mit eigener App-Datenbank (Konten, Passkeys, Verbindungen, Verlauf, Protokoll). Die verwalteten Datenbanken sind Umsysteme: Die Anwendung verbindet sich je Anfrage mit den gespeicherten Angaben, unter Zeit- und Mengengrenzen und nur zu erlaubten Hosts. Mail dient nur für Anmeldecodes und Einladungen.

phpdbadmin — Schichten: eine Oberfläche, vier Dialekte

Was Laravel für alle Treiber kann, nutzen wir; nur die Unterschiede liegen in einem Dialekt je Datenbank



Datenbank-Server bzw. -Dateien

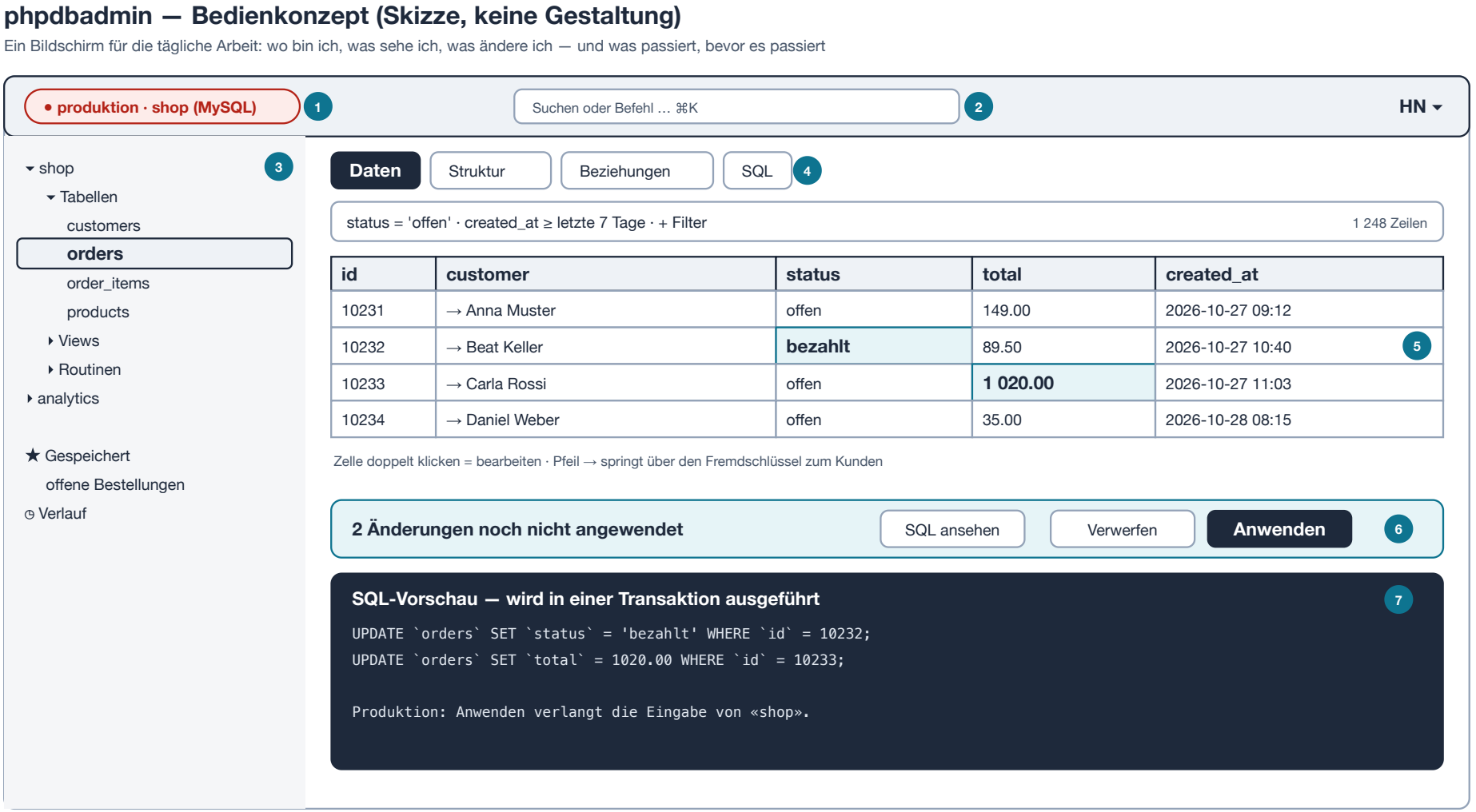
Querschnitt (für jede Anfrage)

Verbindung nur aus gespeicherten Angaben, Hosts nach Liste
Zugangsdaten verschlüsselt (APP_KEY), nie im Browser
Schreibschutz je Verbindung
Zeitlimit je Anweisung
Zeilengrenze je Ergebnis
SQL-Vorschau vor DDL/DML
Protokoll: wer, wann, was
Rechte: lesen / schreiben / verwalten je Verbindung

Beispiel: «Spalte umbenennen» baut der Dienst Struktur mit dem Dialekt der Verbindung; die Oberfläche zeigt das SQL, erst «Anwenden» führt es aus.
SQLite kennt kein ALTER COLUMN — dort baut der Dialekt die Tabelle neu (wie Laravel), die Vorschau zeigt alle Schritte.

Was Laravel für alle Treiber kann (Verbindungen zur Laufzeit, Introspektion, DDL mit Vorschau), wird genutzt; nur die Unterschiede liegen in einer **Dialekt-Schicht** mit einem gemeinsamen Vertrag. Querschnittsregeln (Schreibschutz, Grenzen, Vorschau, Protokoll, Rechte) gelten für jede Anfrage hinter den Controllern, nicht in jedem Controller einzeln.

3.3 Bedienkonzept



1 Verbindung mit Umgebungsfarbe (Produktion rot, schreibgeschützt möglich) · 2 Befehlspalette: Tabelle öffnen, Abfrage, Verbindung wechseln · 3 Navigator mit Gespeichertem und Verlauf
4 gleiche Reiter für jede Tabelle in jeder Datenbank · 5 geänderte Zellen markiert, nichts ist sofort geschrieben · 6 Änderungen sammeln, ansehen, verwerfen · 7 das SQL vor dem Ausführen

Ein Arbeitsbildschirm für alles: Verbindung mit Umgebungsfarbe oben, Befehlspalette, Navigator links, Reiter Daten · Struktur · Beziehungen · SQL, Raster mit markierten Änderungen, Änderungsleiste und SQL-Vorschau. Die Prinzipien P-01 bis P-11 stehen in der Ideensammlung § 6; die Gestaltung (Marke, Farben, Schrift) folgt in der Foundation.

3.4 Nicht im Umfang

SQL Server und Oracle (vorerst) · NoSQL (MongoDB, Redis) · SaaS mit fremden Zugangsdaten · Desktop-App · Berichte und BI · Replikation und Cluster einrichten · Hosting-Panel · Passwort-Anmeldung · Dashboard · Profilseite.

4. Anforderungen

Priorität nach MoSCoW: **M** Must · **S** Should · **C** Could · **W** Won't (diesmal). Stufe nach § 9. Die Abnahme ist so formuliert, dass ein Test oder ein Durchlauf sie belegen kann. Die Verteilung auf Timeboxes mit Must-Anteil unter 60 % je Stufe folgt in der PRL (Foundation).

4.1 A — Anmeldung und Zugang

phpdbadmin — Anmeldung ohne Passwort

Zwei Wege zum selben Konto; das Passwort des Starter-Kits entfällt ganz (A-01 bis A-08)



Neue Konten

- kein «Registrieren» für alle
- erstes Konto: Erst-Setup per Konsole
- weitere: Einladung durch einen Administrator (Link mit Ablauf)
- Konto = E-Mail + Name + Rolle

Schutz

- Code einmalig, gedrosselt je Adresse und IP, gehasht gespeichert
- gleiche Antwort für bekannte und unbekannte Adressen
- Passkeys verwalten: eigener Bereich

Entfernt aus dem Starter-Kit

- Passwort-Anmeldung, Passwort ändern
- Passwort vergessen, Registrierung
- Zwei-Faktor (TOTP), Passwort bestätigen
- E-Mail-Bestätigung (Code bestätigt)
- Dashboard, Profil, Konto löschen

Fortify bleibt nur für die Passkey-Zeremonie; seine Passwort-Endpunkte lehnen ab oder antworten 404 (Muster aus Basalto TB 00, eigenes Projekt).

Nr.	Anforderung	Prio	Stufe	Abnahme
A-01	**Passwort entfernt:** Anmeldung mit Passwort, Registrierung, Passwort vergessen, zurücksetzen, ändern und bestätigen, Zwei-Faktor (TOTP) und E-Mail-Bestätigung des Starter-Kits sind entfernt	M	0	Routenliste ohne diese Routen bzw. Fortify-Endpunkte lehnen ab; Test ruft jede alte Adresse auf (404/405); Spalten `password`, `two_factor_*` entfernt oder ungenutzt
A-02	**Anmeldung mit Code per Mail:** sechs Ziffern, zehn Minuten gültig, einmalig, gehasht gespeichert, gedrosselt je Adresse und IP; gleiche Antwort für bekannte und unbekannte Adressen; Sitzung wird neu erzeugt	M	0	Tests für Erfolg, Ablauf, Wiederverwendung, falschen Code, Drosselung (benannter Limiter), unbekannte Adresse; Sitzungs-ID vor/nach mit Cookie geprüft
A-03	**Anmeldung mit Passkey** und Passkeys verwalten (hinzufügen, umbenennen, entfernen) im Bereich **Sicherheit**	M	0	Zeremonie mit virtuellem Authenticator im Test; einmal von Hand mit Touch ID (Hoang)
A-04	**Dashboard entfernt:** nach der Anmeldung erscheint die Verbindungsübersicht	M	0	`/dashboard` 404; Anmeldung leitet auf die Übersicht
A-05	**Profilseite entfernt:** kein Bearbeiten von Name und E-Mail durch die Person selbst, kein «Konto löschen»; das Kontomenü zeigt Sicherheit (Passkeys, Sitzungen), Darstellung (hell/dunkel, Sprache) und Abmelden	M	0	`/settings/profile` 404; Menü im Durchlauf
A-06	**Keine offene Registrierung:** erstes Konto im Browser mit dem **Einrichtungsschlüssel**, den phpdbadmin als Datei auf dem Server ablegt, oder per Konsole (`php artisan phpdbadmin:install`); danach gleich ein Passkey; weitere Konten per Einladung mit Ablauf durch einen Administrator	M	0	Test: Einladung annehmen, abgelaufene ablehnen; Befehl legt genau ein Administratorkonto an; falscher Schlüssel legt kein Konto an; Einrichtung geschlossen, sobald es ein Konto gibt
A-07	**Rollen und Rechte:** Administrator; Mitglied mit Recht je Verbindung *lesen*, *schreiben* oder *verwalten*	S	2	Policy-Tests je Recht; fremde Verbindung 404 statt 403 (Prüfung vor der Routenbindung)
A-08	Aktive Sitzungen sehen und beenden	C	4	Test: beendete Sitzung ist abgemeldet
A-09	Anmeldungen und Fehlversuche im Protokoll	S	0	Eintrag mit Zeit, Konto bzw. Adresse (gehasht), IP-Prüfsumme

4.2 V — Verbindungen

Nr.	Anforderung	Prio	Stufe	Abnahme
V-01	Verbindung anlegen für **MariaDB, MySQL, PostgreSQL** (Host, Port, Benutzer, Passwort, Datenbank, SSL-Modus) und **SQLite** (Datei in einem erlaubten Ordner)	M	0	je Treiber ein Test gegen den echten Dienst; ungültige Angaben am Feld gemeldet
V-02	**Verbindung testen** vor dem Speichern: Serverversion oder Fehler in Klartext (Host nicht erreichbar, Zugang verweigert, SSL, Datenbank fehlt)	M	0	je Fehlerart ein Test mit erzwungenem Fehler; MySQL/MariaDB nach Code (2002, 1045, 1049), PostgreSQL nach Meldungstext (alle 08006) — F-2
V-03	**Zugangsdaten verschlüsselt** gespeichert und nie an den Browser ausgeliefert; Wahl «Passwort nicht speichern — in der Sitzung fragen»	M	0	Test: Inertia-Props und JSON enthalten kein Passwort; Spalte verschlüsselt (Rohwert ≠ Klartext)

Nr.	Anforderung	Prio	Stufe	Abnahme
V-04	**Umgebung** je Verbindung (lokal, Test, Produktion) mit Farbe in Kopf, Reiter und Seitentitel; Produktion standardmässig schreibgeschützt — Schreibschutz in Schichten (Nur-lesen-Transaktion je Anweisung, vorbereitet, SQLite `OPEN_READONLY`), Lesekonto empfohlen	M	1	Test: schreibgeschützte Verbindung lehnt jede schreibende Anweisung ab, auch die Ausbruchsversuche aus `SecurityProbeTest` — F-7
V-05	**Erlaubte Hosts und SQLite-Ordner** je Installation (Konfiguration); andere werden abgelehnt	M	0	Test mit nicht erlaubtem Host und Pfad (auch `..`, Symlink)
V-06	Verbindungen gruppieren, sortieren, suchen, Favoriten	S	1	Durchlauf
V-07	Herd-Dienste und `.env` der Herd-Sites als Vorschläge erkennen	C	4	Durchlauf auf Hoangs Mac
V-08	SSH-Tunnel	C	4	Test gegen einen lokalen SSH-Dienst
V-09	Verbindungen persönlich oder geteilt (Administrator teilt mit Rechten)	S	2	Test: persönliche Verbindung ist für andere 404
V-10	**Warnung bei Konten mit zu vielen Rechten:** Superuser (PostgreSQL), Dateirechte (`FILE`, `LOAD_FILE` ohne `secure_file_priv`) — sichtbar an der Verbindung, mit Empfehlung eines eigenen Kontos	S	1	Test gegen Herd-`root` auf MariaDB und PostgreSQL (warnt) und ein Lesekonto (warnt nicht) — F-7
V-11	**Schreibschutz auf Zeit aufheben:** eine schreibgeschützte Verbindung lässt sich für 15 Minuten freigeben — nur mit Recht «schreiben», mit Pflichtgrund, im Protokoll, in Kopfleiste und Reiter sichtbar («Schreiben erlaubt bis …»), jederzeit wieder sperrbar	S	2	Test: Freigabe läuft nach 15 Minuten ab (Uhr gestellt), Grund und Ende im Protokoll, ohne Recht 403 — Wireframes 5–7

4.3 O — Oberfläche und Bedienung

Nr.	Anforderung	Prio	Stufe	Abnahme
O-01	**Navigator:** Verbindung → Datenbank → Schema (PostgreSQL) → Objektgruppen → Objekte, mit Suche	M	1	Durchlauf mit 1'000 Tabellen flüssig (N-01); Katalog übergibt das Schema immer ausdrücklich (MariaDB listet sonst alle Datenbanken) — F-1, F-3
O-02	**Befehlspalette ⌘K:** Objekte öffnen, Aktionen ausführen, Verbindung wechseln, zuletzt Benutztes	M	1	Test der Suchquelle; Durchlauf nur mit Tastatur
O-03	**Reiter** für offene Objekte und Abfragen; bleiben beim Neuladen	S	1	Durchlauf
O-04	**Gleiche Bildschirme und Begriffe** für alle Treiber; Besonderheiten mit Erklärung	M	1	Bildschirmfotos derselben Tabelle in allen vier Datenbanken nebeneinander
O-05	Tastatur: Pfeiltasten im Raster, ⌘↵ ausführen, Kürzel-Übersicht	S	1	Durchlauf nur mit Tastatur
O-06	Hell und dunkel; Deutsch und Englisch in Sie-Form	M	1	Sprachdateien vollständig (Test über alle Schlüssel, ohne Rückfall); Kontraste gerechnet
O-07	Auf Tablet und Telefon lesbar, Bearbeiten am Desktop; keine Scrollbalken ausser im Datenraster (P-10)	S	2	Durchlauf bei 375, 768 und 1440 px
O-08	WCAG 2.2 AA in den Kernbildschirmen	S	2	axe ohne Befund; Tastaturdurchlauf
O-09	Leer-, Fehler- und Ladezustände mit Ursache und nächstem Schritt	M	1	Durchlauf mit leerer Tabelle, SQL-Fehler, getrennter Verbindung, abgelaufener Sitzung

Nr.	Anforderung	Prio	Stufe	Abnahme
O-10	Export, Import, Sicherung und Wartung direkt aus dem Menü der Datenbank und der Tabelle — mit Datenbank, Tabelle und Art vorgewählt; Strukturänderungen im selben Menü gesperrt, wenn das Konto nicht schreiben darf	M	3	Test: Vorauswahl aus der Adresse; Durchlauf beider Menüs

4.4 D — Daten

Nr.	Anforderung	Prio	Stufe	Abnahme
D-01	**Datenraster:** blättern (über den Schlüssel, sonst Seitenversatz), Seitengrösse, sortieren, Spalten ein- und ausblenden, Breite merken	M	1	Tests je Treiber mit Sortierung inkl. NULL (Reihenfolge ausdrücklich, nicht treiberabhängig)
D-02	**Filtern** geführt (Spalte, Operator, Wert, und/oder) und als freies WHERE; Suche über die Textspalten der Tabelle, ohne Rücksicht auf Gross- und Kleinschreibung und Umlaute, wo der Treiber es zulässt	M	1	Tests je Treiber, auch mit Umlauten und Werten, die wie SQL aussehen
D-03	Fremdschlüssel folgen (→ Datensatz) und rückwärts (verweisende Zeilen)	S	1	Test je Treiber
D-04	**Bearbeiten:** Zellen ändern, Zeilen einfügen, duplizieren, löschen; Änderungen sammeln; **SQL-Vorschau**; Anwenden in einer Transaktion; Verwerfen	M	2	Test je Treiber: nichts geschrieben vor «Anwenden»; Fehler in der zweiten Anweisung rollt die erste zurück; Vorschau der DML über `pretend()` mit Bindungen; die Vorschau zählt die Zeilen, die über `ON DELETE CASCADE` mitgelöscht würden — F-5, Wireframe 6
D-05	Werte typgerecht: NULL gegen leer, JSON (formatiert und geprüft), Datum und Zeit mit Zeitzone, Binärdaten (Grösse, Herunterladen, Bildvorschau), Enum und Set als Auswahl, Boolean	M	2	Tests je Typ und Treiber, Werte nach dem Speichern byte-genau verglichen
D-06	Zeilen zählen: genau bei kleinen Tabellen, geschätzt bei grossen, «genau zählen» auf Wunsch	S	1	Test mit Schwelle
D-07	Tabellen ohne Primärschlüssel: Bearbeiten nur mit Warnung und eindeutigem Treffer	M	2	Test: mehrdeutige Zeile wird nicht geändert
D-08	Suchen und Ersetzen; Suche über alle Tabellen einer Datenbank	C	3	Test je Treiber
D-09	**Rückgängig für Datenänderungen:** Vorher-Werte der angewendeten Änderungen bis zu einer Grenze gesichert, als Umkehr-SQL wieder anwendbar	S	2	Test: Änderung anwenden, rückgängig, Zustand gleich dem Ausgangszustand

4.5 S — Struktur

Nr.	Anforderung	Prio	Stufe	Abnahme
S-01	**Struktur ansehen:** Spalten (Typ, NULL, Vorgabe, Kommentar, Kollation), Indizes, Fremdschlüssel, Check-Constraints, Trigger; DDL anzeigen und kopieren	M	1	Tests je Treiber gegen eine Beispieltabelle mit allen Merkmalen

Nr.	Anforderung	Prio	Stufe	Abnahme
S-02	**Tabelle anlegen** mit geführtem Formular (Spalten, Primärschlüssel, Indizes, Fremdschlüssel) und SQL-Vorschau	M	2	Test: Vorschau (aus `Blueprint::toSql()`, nie `pretend()`) = ausgeführtes SQL; Tabelle danach per Introspektion gleich der Eingabe — F-5
S-03	**Tabelle ändern:** Spalten hinzufügen, ändern, umbenennen, löschen; Indizes und Fremdschlüssel; Kommentare — angewendet je Treiber (MySQL/MariaDB Anweisung für Anweisung mit Hinweis «nicht rückrollbar», PostgreSQL in einer Transaktion, SQLite nach dem Umbauverfahren)	M	2	Tests je Treiber mit Elterntabelle und abhängigen Zeilen: Daten, Indizes, Fremdschlüssel und Kinder bleiben — F-5
S-04	Tabelle umbenennen, kopieren (Struktur, mit Daten), leeren, löschen — Leeren und Löschen nur mit Namenseingabe	M	2	Test: ohne korrekten Namen keine Ausführung
S-05	Datenbank bzw. Schema anlegen und löschen	S	2	Test je Treiber (SQLite: neue Datei im erlaubten Ordner)
S-06	Views, Routinen, Trigger, Sequenzen, Typen und Enums (PostgreSQL), Events (MySQL/MariaDB) ansehen und bearbeiten	S	3	Test je Objektart und Treiber
S-07	Beziehungsdiagramm aus den Fremdschlüsseln, als SVG exportierbar	S	3	Durchlauf mit Basalto-Datenbank
S-08	Schemavergleich zweier Datenbanken mit Änderungsskript	C	4	Test mit zwei Ständen
S-09	Laravel-Migration aus einer bestehenden Tabelle erzeugen	C	4	Test: Migration erzeugt dieselbe Tabelle auf einem anderen Treiber
S-10	Tabellenoptionen ändern: Speicher-Engine, Zeichensatz und Sortierung, Startwert des Zählers (MySQL/MariaDB) — mit Vorschau	S	3	Test: Vorschau und Anwenden je Option
S-11	Tabelle in eine andere Datenbank bzw. ein anderes Schema desselben Servers kopieren oder verschieben — mit Vorschau	S	3	Test: Kopie mit und ohne Daten, Verschieben, Ziel muss existieren
S-12	Zeichensatz und Sortierung einer Datenbank ändern (MySQL/MariaDB) — mit Vorschau	C	3	Test: Vorschau und Anwenden

4.6 Q — SQL

Nr.	Anforderung	Prio	Stufe	Abnahme
Q-01	**SQL-Editor:** Hervorhebung je Dialekt, Autovervollständigung aus dem Schema, Formatieren	M	1	Durchlauf je Treiber
Q-02	Ausführen: Auswahl, aktuelle Anweisung oder alles; mehrere Ergebnisse als Reiter; Dauer und Zeilenzahl; **Abbrechen**	M	1	Test: Abbrechen beendet die Abfrage auf dem Server; die Oberfläche zeigt «abgebrochen» auch bei MySQL, das ohne Fehler zurückkehrt — F-2
Q-03	**Grenzen:** Zeitlimit je Anweisung (`max_execution_time`, `max_statement_time`, `statement_timeout`), Zeilengrenze je Ergebnis («mehr laden»), schreibgeschützte Verbindungen nur lesend (V-04)	M	1	Tests je Treiber mit langer und mit schreibender Anweisung auf geschützter Verbindung — F-2, F-7

Nr.	Anforderung	Prio	Stufe	Abnahme
Q-04	Verlauf je Konto und Verbindung; gespeicherte Abfragen mit Namen	S	1	Test
Q-05	Ergebnisse bearbeiten, wenn die Abfrage eine Tabelle mit Primärschlüssel liest	C	3	Test
Q-06	EXPLAIN als Baum mit Hinweisen	S	3	Durchlauf je Treiber
Q-07	**Gefährliche Anweisungen** erkennen (DROP, TRUNCATE, DELETE und UPDATE ohne WHERE) und bestätigen lassen	M	2	Tests mit Varianten (Kommentare, Gross-/Kleinschreibung, mehrere Anweisungen)
Q-08	SQL-Hilfe mit KI — nur eingeschaltet, nur mit eigenem Schlüssel, nur das Schema wird gesendet	C	4	Test mit Attrappe: keine Datenzeilen im Auftrag; `Http::preventStrayRequests()`
Q-09	**SQLite-Abfragen aus dem SQL-Editor in einem Arbeitsprozess mit Zeitgrenze** — SQLite kennt kein Zeitlimit je Anweisung und lässt sich in PDO nicht unterbrechen	S	1	Test: eine Abfrage über die Grenze wird beendet, die Oberfläche meldet es — F-2

4.7 E — Export und Import

Nr.	Anforderung	Prio	Stufe	Abnahme
E-01	**Export** von Tabelle, Abfrage oder Datenbank als CSV, JSON, SQL (Struktur, Daten, beides); grosse Exporte im Hintergrund mit Download, in Paketen über den Schlüssel	M	2	Test: Export und Re-Import ergeben dieselben Daten je Treiber; 1 Mio. Zeilen unter 64 MB Speicher (gemessen 3 MB mit `lazyById`) — F-8
E-02	**Import** von CSV mit Spaltenzuordnung und Probelauf; SQL-Datei ausführen mit Fortschritt	M	2	Tests mit Fehlerzeilen; Probelauf schreibt nichts
E-03	Sicherung und Wiederherstellung über die Werkzeuge des Servers (`mariadb-dump`, `mysqldump`, `pg_dump`), wo vorhanden	S	3	Test überspringt sichtbar, wenn das Werkzeug fehlt; Versionsprüfung
E-04	Export als Excel (XLSX)	C	3	Test
E-05	Kopieren aus dem Raster als TSV, Markdown oder INSERT	S	2	Test
E-06	**Statistik aktualisieren** nach eigenen Importen und auf Knopfdruck (`ANALYZE`) — ohne Statistik sortieren MySQL und MariaDB über den Index 100-mal langsamer	S	2	Test: nach Import ist die Schätzung der Zeilenzahl aktuell — F-3

4.8 W — Server-Verwaltung

Nr.	Anforderung	Prio	Stufe	Abnahme
W-01	Server-Übersicht: Version, Laufzeit, Grösse je Datenbank, Zeichensätze	S	3	Test je Treiber
W-02	Prozesse und Sitzungen anzeigen und beenden	S	3	Test: beendete Sitzung ist weg
W-03	Variablen und Status lesen	C	3	Durchlauf
W-04	Datenbank-Benutzer, Rollen und Rechte geführt verwalten (GRANT, REVOKE) mit Vorschau	S	3	Tests je Treiber (ohne SQLite)
W-05	Wartung: ANALYZE, VACUUM (PostgreSQL, SQLite), OPTIMIZE (MySQL, MariaDB), Integritätsprüfung (SQLite)	C	3	Durchlauf

Nr.	Anforderung	Prio	Stufe	Abnahme
W-06	Wartung erweitert: Reparieren (`REPAIR TABLE` , MySQL/MariaDB — nur wo die Engine es kann), Prüfsumme (`CHECKSUM TABLE`), Indizes neu aufbauen (`REINDEX` , PostgreSQL und SQLite)	S	3	Test je Treiber, auch «Engine kann es nicht»

4.9 B — Betrieb und Technik

Nr.	Anforderung	Prio	Stufe	Abnahme
B-01	Stack: Laravel 13, Inertia v3, Vue 3, TypeScript, Tailwind 4; Starter-Kit bereinigt	M	0	Routenliste und Seitenordner ohne Starter-Kit-Reste
B-02	**Dialekt-Schicht:** ein Vertrag, je Treiber eine Umsetzung; Laravel-Introspektion, wo sie reicht	M	0	Vertragstest läuft gegen alle vier Umsetzungen
B-03	**Tests gegen vier Datenbanken** (MySQL 8.4, MariaDB 10.11, PostgreSQL 18, SQLite) lokal; CI sparsam: ein schlanker Auftrag je Push (SQLite), die Matrix wöchentlich und von Hand	M	0	lokales Skript fährt alle vier; Workflow mit `paths-ignore` , `concurrency` , `timeout-minutes`
B-04	**Protokoll** aller schreibenden Aktionen (wer, wann, Verbindung, SQL, betroffene Zeilen) mit Aufbewahrungsfrist; Fehlermeldungen maskiert (Laravel setzt Bindungen in die Meldung ein)	M	2	Test: jede schreibende Aktion erzeugt genau einen Eintrag; Bereinigung nach Frist; ein gebundener Wert erscheint in keiner Meldung — F-7
B-05	Installation per Git oder `composer create-project` und einem Einrichtungsbefehl	S	2	Durchlauf in einem leeren Ordner unter 15 Minuten
B-06	App-Datenbank SQLite als Vorgabe, MariaDB, MySQL und PostgreSQL möglich	M	0	Suite läuft mit jeder App-Datenbank
B-07	Queue für Export und Import; ohne Worker synchron mit Grenze	S	2	Test beider Wege
B-08	Qualität: Pint, Larastan, vue-tsc, `vp check` ; Sicherheitsprüfung vor v1.0	M	0	lokale Prüfkette in der Reihenfolge der CI

4.10 N — Nicht-funktionale Anforderungen

Nr.	Anforderung	Prio	Stufe	Abnahme
N-01	**Leistung:** erste Seite einer Tabelle mit 1 Mio. Zeilen unter 1 s lokal; Navigator mit 1'000 Tabellen flüssig	M	1	Messung mit erzeugten Daten je Treiber, Zeit im Protokoll der Timebox
N-02	**Sicherheit:** keine Zugangsdaten im Browser, CSP, CSRF, Drosselung, Hosts-Liste, Protokoll, maskierte Meldungen; Bedrohungsmodell T-01 bis T-18; White-Box-Prüfung nach OWASP vor v1.0	M	2	Prüfbericht ohne hohen Befund; jede Bedrohung mit Test oder begründet — Sicherheit und Bedrohungen
N-03	**Kein Datenverlust durch die Anwendung:** jede Änderung zuerst als Vorschau; Transaktion, wo möglich; DDL ohne Transaktion (MySQL, MariaDB) ehrlich angezeigt; SQLite-Umbau nie mit aktiven Fremdschlüsseln in einer Transaktion	M	2	Tests mit erzwungenem Fehler mitten in einer Folge und mit Elterntabelle und Kindern (naiv: 0 von 2 Kindern übrig) — F-5
N-04	**Kompatibilität:** MariaDB ab 10.6, MySQL ab 8.0, PostgreSQL ab 14, SQLite ab 3.35 (Annahme R-15)	M	1	Feasibility prüft die Untergrenzen
N-05	Kurze Hilfe in der Oberfläche; Projekt-Doku nach DSDM, bei jedem Commit nachgeführt	S	2	Doku-Prüfung im Commit
N-06	Sprache: Deutsch und Englisch, Sie-Form, Bezeichner im Code englisch	M	1	Test über Sprachdateien

Zählung Fassung 6 (13.11.2026, per Skript): 86 Anforderungen — M 43, S 31, C 12; neu O-10 (M), W-06, S-10, S-11 (S) und S-12 (C), alle nach v1.0.

Zählung (per Skript aus den Tabellen, Fassung 4, 28.10.2026): 81 Anforderungen — M 42, S 28, C 11. Must-Anteil gesamt 52 %; **bis v1.0 (Stufen 0–2, 63 Anforderungen) 67 %** (Fassung 3: 80 und 68 %, Fassung 2: 77 und 71 %); im Plan der PRL mit Rückfallwegen 58 % — über der DSDM-Richtgrösse von 60 %. Die PRL in der Foundation muss deshalb Musts in Kern und Bequemlichkeit teilen oder zurückstufen; Kandidaten mit Rückfallweg «bis dahin über den Navigator bzw. den SQL-Editor»: O-02 Befehlspalette, O-09 Zustände (Kern: Fehler; Rest: Leer- und Ladezustände), D-05 (Kern: NULL, JSON, Datum; Bildvorschau später), E-02 (Kern: CSV; SQL-Datei später), Q-01 (Kern: Hervorhebung und Vervollständigung; Formatieren später).

5. Lösungsansatz und technischer Rahmen

Thema	Ansatz
Rahmen	Laravel 13.39 auf PHP 8.5, Inertia v3, Vue 3, TypeScript, Tailwind 4, Wayfinder (installiert)
Anmeldung	Fortify nur für Passkeys; Code-Anmeldung als eigener kleiner Ablauf (Muster aus Basalto, eigenes Projekt)
Verbindungen	Angaben in der App-Datenbank, Zugangsdaten mit `encrypted`-Cast; Verbindung je Anfrage über `DB::build()`; Sitzungswerte je Treiber (Zeitzone UTC, Zeitlimit, nur lesen)
Katalog	`Schema::getSchemas()`, `getTables()`, `getViews()`, `getTypes()`, `getColumns()`, `getIndexes()`, `getForeignKeys()`; Ergänzungen in der Dialekt-Schicht
Struktur	`Blueprint` mit `toSql()` bzw. `Connection::pretend()` für die Vorschau; dieselben Anweisungen werden danach ausgeführt
Daten	Abfragebau über den Query Builder der Zielverbindung (Bezeichner gequotet, Werte gebunden); Blättern über den Schlüssel
Grosse Daten	`cursor()` / `lazy()` und Queue-Jobs; Dateien mit Ablauf
Oberfläche	eigenes Raster und eigene Bausteine; Editor und Virtualisierung nach Feasibility (Pakete brauchen Freigabe, R-32)
Prüfungen	PHPUnit 12 gegen vier Treiber, Larastan, Pint, vue-tsc, `vp check`; Klickdurchläufe im Browser

6. Geschäftsmodell

Entschieden 28.10.2026 (R-40): Open Source unter MIT, zuerst für den eigenen Gebrauch und als Schaufenster von Nguyen Development; Team-Funktionen später allenfalls als bezahlter Zusatz (Ideensammlung § 9).

7. Erwarteter Nutzen

- **Für Hoang:** ein Werkzeug statt drei; schneller und sicherer in Basalto, Fristo und Kundenprojekten.
- **Für Teams:** Konten statt geteilter Datenbank-Passwörter, Schreibschutz für die Produktion, Protokoll.
- **Für Nguyen Development:** sichtbares Laravel-Produkt mit klarer Nische; Wissen über vier Treiber wird wiederverwendbar.

8. Alternativen

Option	Beschreibung	Bewertung
A — weiter wie bisher	phpMyAdmin, Adminer, Desktop-Client je nach Fall	kein Aufwand; das Problem bleibt, phpMyAdmin auf PHP 8.5 ohne Freigabe
B — Adminer anpassen	Adminer mit eigenem Design und Plugins	schnell; Bedienkonzept (Sammeln, Vorschau, Raster) lässt sich in Adminers Aufbau kaum umsetzen
C — Jaxon DbAdmin einsetzen oder mitbauen	Laravel-App des Mitbewerbers	Frühphase, Kern Adminer, Bootstrap; Richtung nicht in unserer Hand
D — Desktop-Client kaufen	TablePlus o. ä.	gute Bedienung; nicht im Browser, nicht für Teams auf dem Server
E — eigenes Werkzeug (Vorschlag)	phpdbadmin nach diesem CR	volle Kontrolle über Bedienung und Sicherheit; Aufwand rund 40–50 PT; Rückfallposition: nach Stufe 2 vollständig nutzbar

9. Aufwand und Stufen

phpdbadmin — Stufen und Meilensteine

Reihenfolge, ohne Termine. Jede Stufe ist für sich nutzbar; phpdbadmin läuft neben Basalto, das bis zur Beta Vorrang hat (R-03)



Summe rund 40–50 Personentage bis v1.2 (Schätzung vor Feasibility). Rückfallposition: nach Stufe 2 ist das Werkzeug vollständig nutzbar — Verwalten und Team folgen.

Stufe	Inhalt	Schätzung	Meilenstein
0 Fundament	Starter-Kit bereinigt, Passkey und Code, Verbindungen, Dialekt-Schicht, Tests auf vier Datenbanken	6–8 PT	—
1 Lesen	Navigator, Raster, Filter, Struktur ansehen, SQL-Editor, Befehlspalette	8–10 PT	**Alpha** — lesend im eigenen Alltag
2 Bearbeiten	Bearbeiten mit Vorschau und Rückgängig, Struktur ändern, Export und Import, Rechte, Protokoll	10–12 PT	**v1.0** — ersetzt phpMyAdmin und phpPgAdmin
3 Verwalten	Benutzer und Rechte, Prozesse, Views, Routinen, Beziehungsdiagramm, EXPLAIN, Sicherung	8–10 PT	**v1.1**
4 Komfort und Team	Schemavergleich, Laravel-Migration, Herd, SSH-Tunnel, Sitzungen, KI optional	8–10 PT	**v1.2**

Summe rund 40–50 PT vor der Feasibility; mit ihren Auflagen (Arbeitsprozess für SQLite, Anwenden je Treiber, Kontowarnung, Maskierung) **rund 42–53 PT** (Feasibility § 14).

10. Risiken auf oberster Ebene

Nr.	Risiko	Wirkung	Gegenmittel
R1	Zeit neben Basalto und Fristo	Stufen verschieben sich	Stufen einzeln nutzbar; Vorrang entschieden: neben Basalto, das bis zur Beta vorgeht (R-03)
R2	Treiberunterschiede grösser als erwartet (Typen, DDL, Rechte)	Aufwand je Funktion ×4	Dialekt-Schicht, Laravel-Introspektion zuerst, Feasibility-Probe auf allen vier
R3	Sicherheit: ein Werkzeug mit Zugang zu allen Datenbanken ist ein lohnendes Ziel	Datenabfluss	Sicherheit ab Werk (N-02), Hosts-Liste, Protokoll, White-Box-Prüfung
R4	Leistung des Rasters bei grossen Tabellen	träge Oberfläche	Messung in der Feasibility, Blättern über den Schlüssel, Virtualisierung
R5	Mitbewerber (Jaxon DbAdmin, phpMyAdmin 6)	Nische schrumpft	Bedienung als Unterscheidung, früher Alpha-Meilenstein
R6	Name zu nahe an phpMyAdmin (bewusst gewählt, R-04)	Einspruch des phpMyAdmin-Projekts, Verwechslung	Markenrecherche vor dem ersten öffentlichen Auftritt, eigenständiger Auftritt, kein Hinweis auf eine Verbindung zu phpMyAdmin (F-10)
R7	Arbeitsvertrag (Nebenbeschäftigung)	Veröffentlichung blockiert	Meldung vor dem ersten öffentlichen Auftritt, wie bei Basalto (R-05)

11. Rahmen und Abgrenzung

Träger (entschieden 28.10.2026, R-01): Nguyen Development (Einzelfirma von Hoang Nguyen); Lizenz MIT (R-40); Betrieb lokal und selbst gehostet, kein SaaS (R-11). Kein Code, keine Daten und keine Texte aus Arbeitgeber-Projekten (`~/Development/uzh-*`); aus Basalto und Fristo (eigene Projekte) darf übernommen werden. Code von phpMyAdmin, phpPgAdmin und Adminer wird nicht übernommen.

12. Abhängigkeiten und Voraussetzungen

- **MySQL in Herd** installieren (Version 8.4, eigener Port, z. B. 3307), damit die Testmatrix vollständig ist (R-33).

- **Mail lokal** über Herd (SMTP 127.0.0.1:2525) für Anmeldecodes.
- `APP_URL` auf `https://phpdbadmin.test` stellen (heute `http://localhost:8000`) — Passkeys hängen an Herkunft und Relying Party.
- **Neue Pakete** (SQL-Editor, Raster-Virtualisierung) nur nach Freigabe (R-32).
- **Repo und Actions:** GitHub-Repo noch nicht angelegt; der vorhandene Workflow `tests.yml` braucht `paths-ignore`, `concurrency` und `timeout-minutes` nach den Regeln für sparsame Actions.

13. Offene Punkte

Alle offenen Punkte stehen mit Annahme im Rückfragen-Katalog (R-01 bis R-42).

14. Entscheid und nächste Schritte

Schritt	Inhalt	Stand
1	Rückfragen beantworten (mindestens R-01, R-03, R-04, R-11, R-40)	✅ 28.10.2026; übrige laufen mit Annahme
2	Entscheid **Weiterführen** auf Grundlage dieses CR	✅ 28.10.2026 (Hoang: «Feasibility starten»)
3	Feasibility mit den Prüfpunkten F-1 bis F-10 (02-feasibility)	✅ 28.10.2026 — 6 bestanden, 4 bedingt, Empfehlung Weiterführen
4	Terms of Reference v1.0 nach Feasibility	✅ verabschiedet 28.10.2026
5	Foundation: Gestaltung, PRL, Architektur, Datenmodell, Timebox-Plan; danach TB 00 Fundament	🔨 begonnen 28.10.2026

Anhang A — Ideensammlung — phpdbadmin

Stand: 28.10.2026 · v0.1 — Entwurf mit Annahmen, lebendes Dokument der Ideenphase. Offene Punkte tragen eine Rückfrage-Nummer aus Ausgangslage und Rückfragen; bis zur Antwort gilt die Annahme dort.

1. Ausgangslage

1.1 Auslöser

Hoang am 28.10.2026: «Ein Admin-Tool für verschiedene Datenbanken — MariaDB, MySQL, PostgreSQL und SQLite. Wie phpMyAdmin oder phpPgAdmin, aber intuitiver und benutzerfreundlicher, und es soll verschiedene Plattformen bedienen können.» Dazu drei feste Vorgaben für das Laravel-Starter-Kit: **kein Passwort** (nur Passkey und Code per Mail), **kein Dashboard**, **kein Profil**. Vorgehen und Dokumentation wie bei Fristo und Basalto.

Ausgangspunkt im Repo ist ein frisches Laravel-13-Starter-Kit (Vue) mit Laravel Boost (Commits `a7a2127` bis `fa95ab3`). Unter `~/Development` liegen Kopien von phpMyAdmin 5.2.2 und phpPgAdmin 7.13.0 — sie dienen als Anschauung für den Funktionsumfang; ihr Code wird nicht übernommen (GPL, und phpdbadmin baut seine Oberfläche ohnehin neu).

1.2 Name

phpdbadmin ist der Name — entschieden von Hoang am 28.10.2026 (R-04), nach der Prüfung unten und gegen meinen Vorschlag einer Namensfindung. Er sagt sofort, was gemeint ist: «wie phpMyAdmin, aber für alle Datenbanken», und wird so auch gesucht.

Prüfung	Befund (28.10.2026)
Domain `phpdbadmin.ch`	frei (SWITCH-Whois)
Domain `phpdbadmin.com`	belegt seit 30.08.2025 (Inhaber nicht geprüft)
Gleichnamige Software	keine gefunden (Websuche); ähnlich: «PHP Data Admin» (SourceForge, seit 2015 ruhend)
Sprechtest	«pe-ha-pe-de-be-admin» — sieben Silben, am Telefon nur buchstabiert verständlich (Lehre aus der Namensfindung von Fristo: erst sprechen, dann suchen)
Verwechslung	sehr nahe an «phpMyAdmin» — hilft beim Finden, kann aber Markenrechte des phpMyAdmin-Projekts berühren (zu prüfen)
Aussage	nennt die Technik (PHP), nicht den Nutzen; Kundinnen und Kunden interessiert PHP nicht

Mein Vorschlag war, den Namen nur als Arbeitstitel zu führen und vor dem ersten öffentlichen Auftritt eine Namensfindung zu machen. **Entscheid 28.10.2026: phpdbadmin ist der Name.** Folgen: `phpdbadmin.ch` sichern (Hoang); vor dem ersten öffentlichen Auftritt eine Markenrecherche (IGE, EUIPO) zur Nähe zu «phpMyAdmin» und ein eigenständiger Auftritt (Logo, Farben), der nicht wie phpMyAdmin aussieht (Prüfpunkt F-10).

1.3 Bezug zu den anderen Vorhaben

Basalto (CMS, läuft auf SQLite, MariaDB, MySQL und PostgreSQL), Fristo (PostgreSQL) und die Kundenprojekte brauchen alle einen Blick in ihre Datenbanken. Heute geschieht das mit wechselnden Werkzeugen. phpdbadmin wäre das gemeinsame Werkzeug — und zugleich das Projekt, in dem das in Basalto gesammelte Wissen über Treiberunterschiede (NULL-Sortierung, JSON-Schlüsselordnung in MySQL, Collations, `TEXT`-Grenze 64 KB, `timestamp`-Fallen in MariaDB 10.6) zum Produkt wird.

2. Warum — Problem und Lücke

phpdbadmin — Problembild: heute und mit phpdbadmin

Wer MariaDB, MySQL, PostgreSQL und SQLite im Browser verwalten will, braucht heute mehrere Werkzeuge — keines führt durch die Arbeit

HEUTE

phpMyAdmin 5.2.3

nur MySQL und MariaDB
offiziell nur PHP < 8.4
Version 6 seit Jahren
in Entwicklung
viele Reiter und Formulare
Anmeldung mit dem
Datenbank-Passwort

phpPgAdmin 7.13

nur PostgreSQL
seit Dezember 2023
ohne Commit
offene XSS-Meldung 2026
neuere PostgreSQL-
Versionen nur in Forks
Oberfläche der 2000er

Adminer 6.2

alle vier, eine PHP-Datei
aktiv gepflegt
knapp und schnell
Bedienung über Links
und Formulare
wenig Führung, keine
Vorschau vor dem Ändern

Desktop-Clients

TablePlus, DBeaver,
Beekeeper, HeidiSQL
je Rechner installiert
nicht im Browser,
nicht auf dem Server
teils kostenpflichtig
oder mit Tab-Grenzen

Gemeinsam: keine einheitliche Bedienung über vier Datenbanken · Änderungen ohne Vorschau des SQL · kein Schutz vor DROP auf der Produktion · Anmeldung mit Datenbank-Passwörtern statt mit eigenem Konto

MIT PHPDBADMIN

phpdbadmin

eine Oberfläche für alle vier Datenbanken
Tabelle bearbeiten wie in einer
Tabellenkalkulation
SQL-Vorschau vor jedem Schreiben
Befehlspalette ⌘K für alles
Produktion farbig und schreibgeschützt
Anmeldung mit Passkey oder Code
Zugangsdaten verschlüsselt gespeichert
Laravel 13, Inertia v3, Vue 3, Tailwind 4
läuft, wo Laravel läuft (Herd, Server)

Was sich ändert

Entwickler	Lokale Datenbanken aus Herd in einer Oberfläche, egal ob MariaDB oder PostgreSQL; SQL lernen am gezeigten SQL
Betrieb / Admin	Ein Werkzeug auf dem Server statt phpMyAdmin und phpPgAdmin; Konten mit Passkey, Protokoll, Schreibschutz
Nguyen Development	Eigenes Werkzeug für alle Projekte (Basalto, Fristo, Kunden); Schaufenster für Laravel-Handwerk

2.1 Das Problem

1. **Ein Werkzeug je Datenbank.** phpMyAdmin kann nur MySQL und MariaDB, phpPgAdmin nur PostgreSQL, für SQLite braucht es ein drittes. Wer wie Hoang alle vier nutzt, wechselt Oberfläche, Begriffe und Gewohnheiten.
2. **Bedienung aus einer anderen Zeit.** Ein Formular je Datensatz, viele Reiter, Bestätigungen ohne Inhalt («Wirklich ausführen?»). Die Werkzeuge zeigen Tabellen, aber sie führen nicht durch die Arbeit.
3. **Ändern ohne Netz.** Ein Klick schreibt sofort; es gibt keine Vorschau des SQL, kein Sammeln von Änderungen, kein Rückgängig und kein Protokoll. Die Produktion sieht aus wie die lokale Datenbank.
4. **Anmeldung mit Datenbank-Passwörtern.** phpMyAdmin und Adminer melden direkt mit Datenbank-Zugangsdaten an; öffentlich erreichbare Installationen sind ein bekanntes Angriffsziel.
5. **Technischer Stillstand.** phpMyAdmin 5.2.3 (Oktober 2025) läuft offiziell nur bis PHP 8.3, Version 6 ist seit Jahren in Entwicklung; phpPgAdmin hat seit Dezember 2023 keinen Commit mehr und eine offene XSS-Meldung von 2026 (Marktbild § 1). Herd und neue Server laufen mit PHP 8.4 und 8.5.

2.2 Die Lücke (These, im Marktbild geprüft)

Ein Werkzeug **im Browser**, das **alle vier Datenbanken mit derselben Bedienung** abdeckt, **geführt** arbeitet (Vorschau, Sammeln, Schutz), **auf dem Laravel-Stack** läuft (Herd, jeder PHP-Server) und **eigene Konten ohne Passwort** hat. Adminer deckt die Breite ab, aber nicht die Führung; DbGate und CloudBeaver haben moderne Oberflächen, aber nicht PHP; Jaxon DbAdmin (seit Oktober 2026 sichtbar) zielt in dieselbe Richtung, steht aber bei Version 0.35 auf dem Kern von Adminer mit Bootstrap-Oberfläche (Marktbild § 2).

2.3 Warum wir

- Laravel 13 bringt die Mehrdatenbank-Fähigkeit schon mit: Verbindungen zur Laufzeit (`DB::build()`, `DB::connectUsing()`), Introspektion für alle Treiber (`Schema::getSchemas()`, `getTables()`, `getViews()`, `getTypes()`, `getColumns()`, `getIndexes()`, `getForeignKeys()`), DDL-Erzeugung mit Vorschau (`Blueprint::toSql()`, `Connection::pretend()`) — geprüft im Quellcode von Laravel 13.39 am 28.10.2026.
- Die passwortlose Anmeldung mit Code und Passkey ist in Basalto gebaut und erprobt (eigenes Projekt, darf übernommen werden).
- Die Lektionen aus Basalto und Fristo zu den Unterschieden der vier Treiber sind genau das Wissen, das ein Datenbank-Werkzeug braucht.

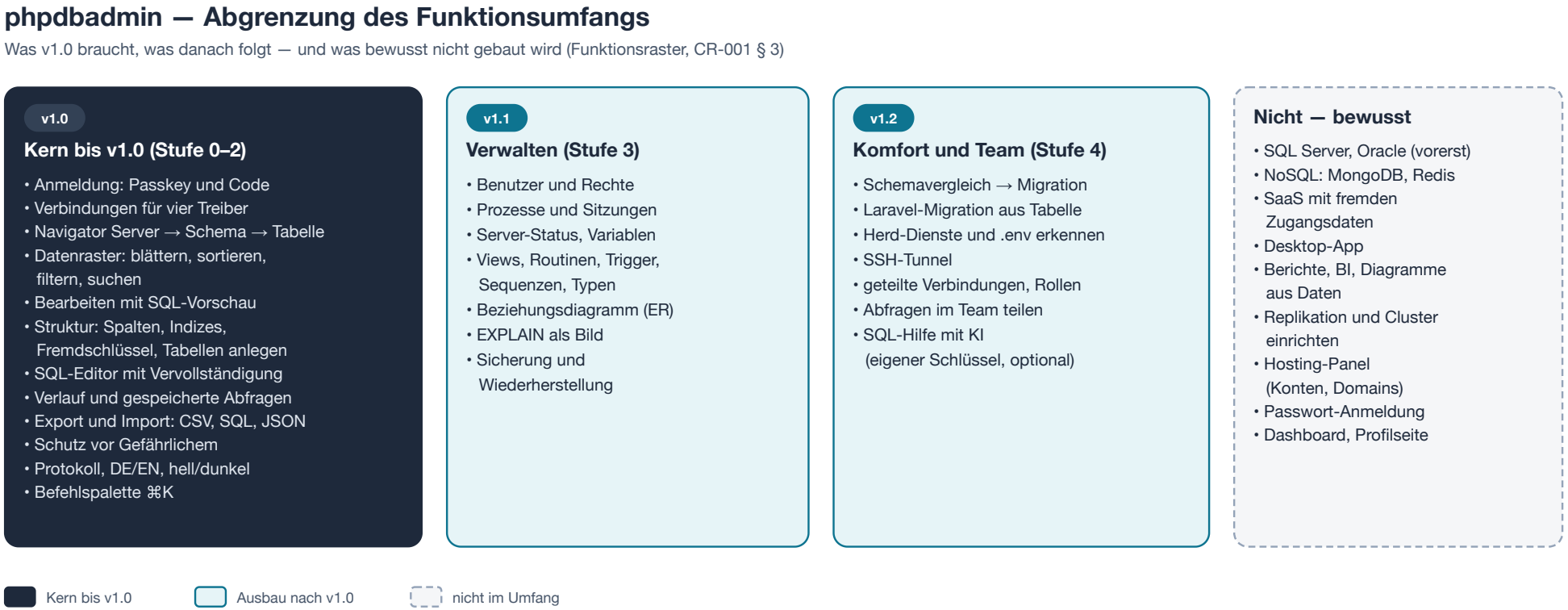
3. Für wen — Zielgruppen und Rollen

Gruppe	Situation	Was sie braucht
Laravel- und PHP-Entwickler (zuerst, Annahme R-10)	lokal in Herd, MariaDB und PostgreSQL nebeneinander, SQLite in Tests	schnell nachsehen, filtern, eine Zeile korrigieren, Struktur prüfen, SQL ausprobieren
Betrieb in kleinen Teams	phpMyAdmin auf dem Server, mehrere Personen	Konten statt geteilter DB-Passwörter, Rechte je Verbindung, Protokoll, Schreibschutz für die Produktion
Fachleute mit Datenpflege	pflegen Stammdaten, exportieren für Excel	geführte Bearbeitung, verständliche Fehlermeldungen, Export ohne SQL

Gruppe	Situation	Was sie braucht
Nguyen Development	Basalto, Fristo, Kundenprojekte	ein Werkzeug für alle Projekte; Schaufenster für Laravel-Handwerk

Rollen in der Anwendung (Vorschlag): Administrator (Konten, Verbindungen, Einstellungen) · Mitglied mit Recht je Verbindung: *lesen*, *schreiben* oder *verwalten* (Struktur, Benutzer und Rechte des Servers).

4. Was — Funktionsumfang



Kern bis v1.0 sind Anmelden, Verbinden, Lesen, Bearbeiten und Austauschen; danach folgen die Server-Verwaltung (v1.1) sowie Komfort- und Team-Funktionen (v1.2). Die vollständige Einordnung je Funktion mit dem Vergleich zu phpMyAdmin, phpPgAdmin und Adminer steht im Funktionsraster, die Anforderungen mit Abnahme in CR-001 § 4.

5. Was nicht — Abgrenzung (Vorschlag)

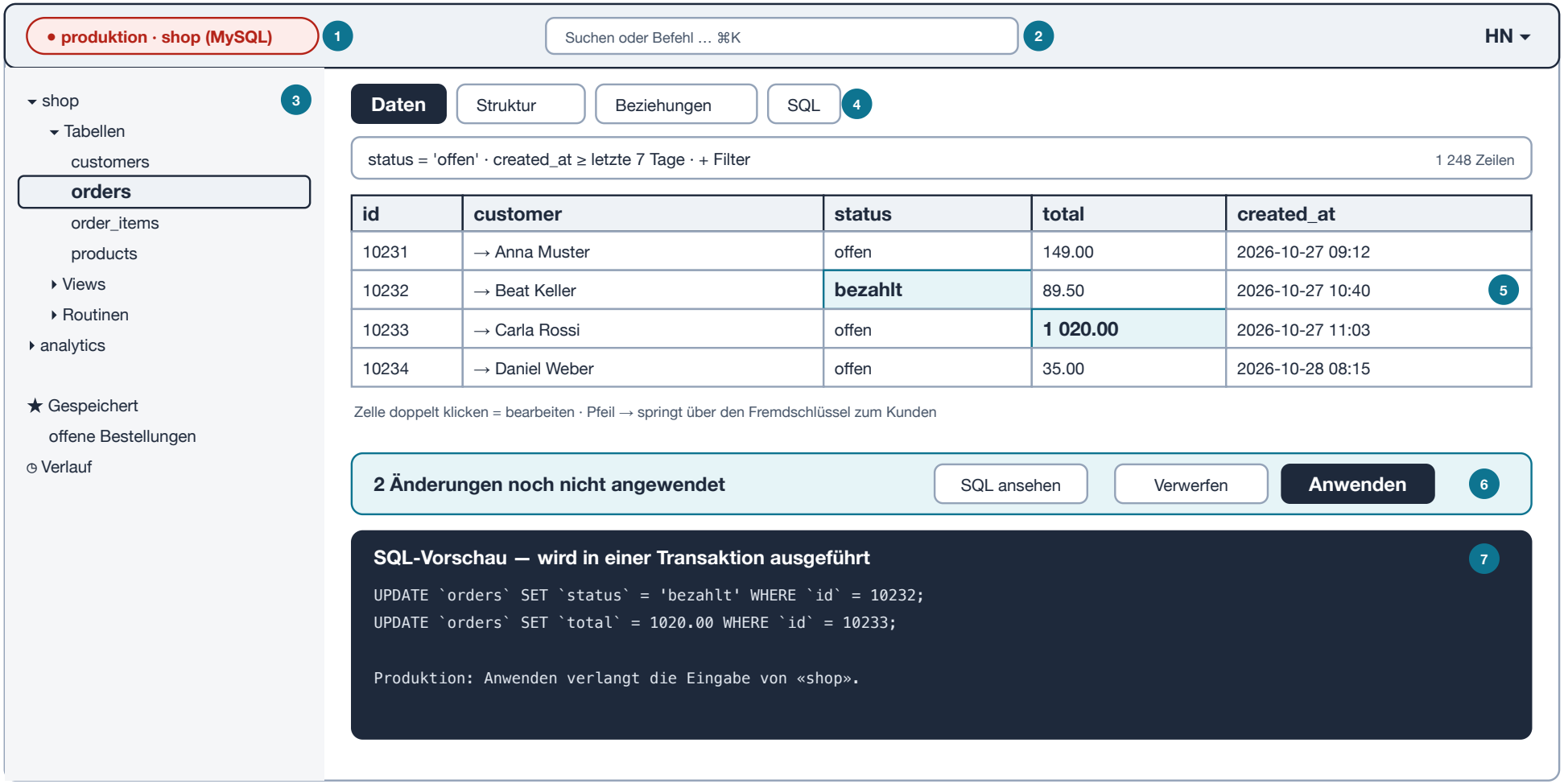
- **Weitere Datenbanken** (SQL Server, Oracle, NoSQL wie MongoDB und Redis) — nicht vor v1.2; die Dialekt-Schicht hält die Tür offen (R-14).
- **SaaS mit fremden Zugangsdaten** — phpdbadmin wird lokal oder selbst gehostet betrieben; Zugangsdaten fremder Datenbanken auf einem eigenen Server zu halten, wäre ein anderes Produkt mit anderem Risiko (R-11).
- **Desktop-App**, **Berichte und BI** (Diagramme aus Daten), **Replikation und Cluster einrichten**, **Hosting-Panel** (Konten, Domains wie cPanel).
- **Passwort-Anmeldung, Dashboard, Profilseite** — ausdrücklich entfernt (Vorgabe).

6. Wie — Produktprinzipien (Vorschlag)

Was «intuitiver und benutzerfreundlicher» heisst, legen diese Prinzipien fest. Jedes ist im Bedienkonzept sichtbar und in CR-001 § 4 mit einer Abnahme verknüpft.

phpdbadmin — Bedienkonzept (Skizze, keine Gestaltung)

Ein Bildschirm für die tägliche Arbeit: wo bin ich, was sehe ich, was ändere ich — und was passiert, bevor es passiert



1 Verbindung mit Umgebungsfarbe (Produktion rot, schreibgeschützt möglich) · 2 Befehlspalette: Tabelle öffnen, Abfrage, Verbindung wechseln · 3 Navigator mit Gespeichertem und Verlauf
4 gleiche Reiter für jede Tabelle in jeder Datenbank · 5 geänderte Zellen markiert, nichts ist sofort geschrieben · 6 Änderungen sammeln, ansehen, verwerfen · 7 das SQL vor dem Ausführen

Nr.	Prinzip	Was es bedeutet
P-01	**Eine Sprache für vier Datenbanken**	gleiche Begriffe und Bildschirme; Unterschiede nur, wo sie fachlich zählen (Schemas in PostgreSQL, Engines in MySQL), dann mit Erklärung
P-02	**Nichts passiert ungesehen**	jede schreibende Aktion zeigt ihr SQL vor der Ausführung; Änderungen werden gesammelt und einmal angewendet — in einer Transaktion, wo der Treiber es kann (DDL in MySQL und MariaDB nicht: das wird ehrlich angezeigt)
P-03	**Direkt bearbeiten statt Formular**	Raster wie eine Tabellenkalkulation; lange Inhalte (JSON, Text, Binärdaten) in einer Seitenschublade, inline und ohne Fenster in der Schublade
P-04	**Gefahr sichtbar machen, nicht verbieten**	Umgebungsfarbe je Verbindung, Schreibschutz-Schalter, Namenseingabe für DROP und TRUNCATE, betroffene Zeilen vor UPDATE und DELETE zählen, Rückgängig für Datenänderungen
P-05	**Tastatur zuerst**	Befehlspalette ⌘K, Ausführen mit ⌘↵, Pfeiltasten im Raster
P-06	**Schnell auch bei grossen Tabellen**	nie unbegrenzt lesen; Blättern über den Schlüssel, geschätzte Zeilenzahl, Abbrechen langer Abfragen, Export im Hintergrund
P-07	**Zustände erklären**	leere Tabelle, Fehler, Laden — jeweils mit Ursache in Klartext und dem nächsten Schritt
P-08	**Lernen nebenbei**	das gezeigte SQL ist kopierbar und lässt sich im Editor öffnen
P-09	**Sicher ab Werk**	kein Passwort, verschlüsselte Zugangsdaten, erlaubte Hosts, Protokoll, CSP
P-10	**Keine Scrollbalken, wo vermeidbar**	Regel von Hoang (15.10.2026); das Datenraster ist die begründete Ausnahme für waagrechtes Scrollen, mit fixierter erster Spalte und Verlaufskante (R-25)
P-11	**Deutsch und Englisch, Sie-Form**	Oberfläche zweisprachig ab v1.0

7. Womit — Technik (Vorschlag)

Baustein	Vorschlag	Stand
Rahmen	Laravel 13.39 (PHP 8.5), Inertia v3.5, Vue 3, TypeScript, Tailwind 4, Vite (vite-plus), Wayfinder	installiert
Anmeldung	Fortify 1.41 nur für Passkeys (enthält `laravel/passkeys`), eigener Code-Login per Mail	installiert, umzubauen in Stufe 0
Zielverbindungen	Konfiguration aus der App-Datenbank, Verbindung je Anfrage mit `DB::build()`, Grenzen je Treiber (Zeitlimit, nur lesen)	Prüfpunkt Feasibility
Katalog	Laravel-Schema-Introspektion, ergänzt durch eine **Dialekt-Schicht** je Treiber für Prozesse, Rechte, Routinen, EXPLAIN, Server-Werte	Prüfpunkt Feasibility
Struktur ändern	`Blueprint` + `toSql()` bzw. `pretend()` für die SQL-Vorschau; SQLite-Tabellenumbau wie Laravel	Prüfpunkt Feasibility
SQL-Editor	CodeMirror 6 mit SQL-Dialekten (MIT)	probeweise installiert 28.10.2026 (Freigabe Hoang), F-4 bestanden
Datenraster	eigenes Raster mit `@tanstack/vue-virtual` (MIT), Blättern über den Schlüssel	probeweise installiert, F-3 bestanden
App-Datenbank	SQLite als Vorgabe (Konten, Passkeys, Verbindungen, Verlauf, Protokoll)	Annahme R-31
Hintergrund	Queue (`database`) für Export und Import	vorhanden

Baustein	Vorschlag	Stand
Tests	PHPUnit 12 gegen MySQL 8.4, MariaDB 10.11, PostgreSQL 18 und SQLite	MySQL 8.4.11 in Herd seit 28.10.2026; Feasibility-Proben laufen auf allen vier

Die Schichten im Detail: Architektur-Diagramm in CR-001 § 3.2.

8. Wo — Betrieb und Umgebungen

Stufe	Ort	Bemerkung
Dev	`https://phpdbadmin.test`	Laravel Herd (macOS), HTTPS aktiv — Passkeys brauchen einen sicheren Ursprung; MySQL 8.4.11 :3307, MariaDB 10.11.6 :3306, PostgreSQL 18 :5432
Testing / Demo	offen	Demo mit mitgelieferten SQLite-Beispieldatenbanken denkbar (R-30)
Produktion	beim Anwender	selbst gehostet neben den eigenen Datenbanken; kein SaaS (entschieden 28.10.2026, R-11)

Ein Hinweis zur Cloud: Ein Datenbank-Werkzeug muss die Datenbanken erreichen, und die liegen meist in internen Netzen. Laravel Cloud eignet sich deshalb höchstens für eine Demo, nicht als Betriebsort des Produkts.

9. Wieviel — Geschäftsmodell (offen)

Der Markt ist von kostenloser Software geprägt (phpMyAdmin, Adminer, pgAdmin), daneben Open Core (DbGate, Beekeeper Studio, CloudBeaver) und bezahlte Desktop-Werkzeuge (TablePlus, DataGrip) — Marktbild § 4. Drei Wege:

Option	Beschreibung	Einschätzung
A — Open Source (MIT)	frei, Schaufenster für Nguyen Development	passt zum Markt; kein direkter Erlös
B — Open Core	Kern frei, Team-Funktionen (Rollen, geteilte Verbindungen, Protokoll-Export, SSO) bezahlt	Erlös möglich, aber Abgrenzung pflegen
C — kommerziell wie Basalto	Lizenz je Installation	schwer gegen kostenlose Platzhirsche

Entschieden 28.10.2026: A — Open Source unter MIT, Träger Nguyen Development (R-01, R-40). Team-Funktionen lassen sich später als B anbieten. Folge: Abhängigkeiten müssen MIT-verträglich sein; Code aus GPL-Projekten (phpMyAdmin, phpPgAdmin) kommt ohnehin nicht hinein.

10. Wann — Fahrplan (Grobvorschlag)



Summe rund 40–50 Personentage bis v1.2 (Schätzung vor Feasibility). Rückfallposition: nach Stufe 2 ist das Werkzeug vollständig nutzbar — Verwalten und Team folgen.

Rund 40–50 Personentage bis v1.2, Schätzung vor der Feasibility. **Entschieden 28.10.2026 (R-03):** phpdbadmin läuft in kleinen Timeboxes neben Basalto, das bis zur Beta (Anfang 2027) Vorrang behält; Stufen 0–1 bis zur Alpha zuerst.

11. Wer — Rollen

Hoang Nguyen als Business Sponsor, Visionary, Projektleitung und Technical Coordinator in Personalunion; Claude Code als Pair. Business Ambassador: Hoang selbst als erster Nutzer, später ein Laravel-Entwickler aus dem Umfeld als Gegenleser (R-02).

12. Woran gemessen — Erfolgskriterien (Vorschlag)

- Hoang ersetzt phpMyAdmin, phpPgAdmin und den Desktop-Client im eigenen Alltag ab v1.0.
- Häufige Aufgaben (Tabelle öffnen, filtern, Zeile ändern, exportieren) in höchstens drei Klicks oder zwei Tastenkürzeln.
- Dieselben Abläufe sind auf vier Datenbanken automatisch getestet (MySQL 8.4, MariaDB 10.11, PostgreSQL 18, SQLite).
- Die erste Seite einer Tabelle mit einer Million Zeilen erscheint lokal in unter einer Sekunde.
- Eine White-Box-Sicherheitsprüfung nach OWASP vor v1.0 ohne hohen Befund.
- Kernbildschirme ohne Befund nach WCAG 2.2 AA.

13. Offene Entscheide

Alle offenen Punkte stehen mit Annahme im Rückfragen-Katalog; entschieden sind am 28.10.2026 Träger und Lizenz (R-01, R-40), Betriebsmodell (R-11), Vorrang (R-03) und Name (R-04).

Anhang B — Ausgangslage und Rückfragen — phpdbadmin

Stand: 28.10.2026, erste Runde mit interaktiven Antworten zu R-01, R-03, R-04, R-11 und R-40 (Abschnitt 4). Dieses Dokument sammelt, was feststeht, was ich selbst nachgelesen habe und was nur Hoang beantworten kann. Jede Rückfrage trägt eine **Annahme**, die gilt, solange nichts dagegen spricht. Die wichtigsten Fragen werden interaktiv gestellt (Optionen zum Anklicken, Empfehlung zuerst); beantwortete Fragen bekommen das Datum der Antwort. Aus den Antworten entsteht die nächste Fassung von CR-001.

1. Was feststeht

Thema	Festlegung	Quelle
Vorhaben	Admin-Werkzeug für **MariaDB, MySQL, PostgreSQL und SQLite** wie phpMyAdmin oder phpPgAdmin, aber **intuitiver und benutzerfreundlicher**, für alle vier Plattformen	Hoang 28.10.
Rahmen	**Laravel-Projekt** im Repo `~/Development/phpdbadmin`, Herd-Site `phpdbadmin.test`	Hoang 28.10.
Starter-Kit	**Passwort entfernen** — nur Passkey und Code per Mail («Magic Code») · **Dashboard entfernen** · **Profil entfernen**	Hoang 28.10.
Dokumentation	in `dokumentationen/` nach DSDM wie Fristo und Basalto; Beginn mit dem Pre-Project	Hoang 28.10.
Sprache	Deutsch mit echten Umlauten in Doku, Kommentaren und Commits	globale Regeln
Arbeitsweise	Rückfragen interaktiv mit Empfehlung zuerst; Defaults ohne Rückfrage, nachfragen nur bei hohem Blast-Radius	globale Regeln, Lektionen
GitHub Actions	sparsam: ein schlanker Auftrag je Push, Schweres wöchentlich oder von Hand, Doku ohne Lauf	globale Regeln (25.10.2026)
Träger und Lizenz	**Produkt von Nguyen Development, Open Source unter MIT**	R-01, R-40
Betriebsmodell	**lokal (Herd) und selbst gehostet** auf eigenen Servern, mehrere Konten mit Rechten; **kein SaaS**	R-11
Vorrang	**neben Basalto** in kleinen Timeboxes; Basalto behält Vorrang bis zur Beta; Stufen 0–1 zuerst	R-03
Name	**phpdbadmin**, endgültig	R-04

2. Was ich selbst nachgelesen habe (keine Rückfrage nötig)

Frage	Antwort	Quelle
Was bringt das Starter-Kit mit?	Routen `home`, `dashboard` (auth + verified), `settings/profile`, `settings/security` (mit `RequirePassword`), `settings/password`, `settings/appearance`, `.well-known/passkey-endpoints`; Seiten `Welcome`, `Dashboard`, `auth/*` (Login, Register, ForgotPassword, ResetPassword, ConfirmPassword, TwoFactorChallenge, VerifyEmail), `settings/*` (Profile, Security, Appearance); Fortify mit Registrierung, Passwort-Reset, E-Mail-Bestätigung, Zwei-Faktor und **Passkeys** (`confirmPassword: true`) — gleiche Lage wie Basalto vor TB 00	`routes/web.php`, `routes/settings.php`, `resources/js/pages`, `config/fortify.php`
Welche Versionen sind installiert?	Laravel 13.39.0 · Fortify 1.41.0 (bringt `laravel/passkeys` mit) · Inertia Laravel 3.5.2 · Wayfinder 0.1.23 · PHPUnit 12.5.42 · Larastan 3.13.2 · Pint 1.32.1 · Boost 2.10.3; npm: `@inertiajs/vue3` 3, `@laravel/passkeys` 0.2, Vue 3.5, Tailwind 4, `vite-plus` 1.4.0	`composer show --direct`, `package.json`
Welche Datenbanken laufen lokal?	Herd: **MariaDB 10.11.6 :3306**, **PostgreSQL 18 :5432**, SQLite nativ; MySQL war nicht installiert — seit 28.10.2026 **MySQL 8.4.11 :3307** (R-33, Freigabe Hoang)	Herd `services list` 28.10.2026
Wie ist die Site erreichbar?	`https://phpdbadmin.test` (Herd: gesichert, PHP 8.5, Node 24); `APP_URL` in der `.env` steht noch auf `http://localhost:8000`, `APP_NAME=Laravel`, `APP_LOCALE=en`, App-Datenbank SQLite	Herd `site info`, `.env` (nur Schlüsselnamen gelesen)
Was kann Laravel 13 schon für mehrere Datenbanken?	Verbindungen zur Laufzeit: `DB::build()`, `DB::connectUsing()`; Introspektion für alle Treiber: `Schema::getSchemas()`, `getTables()`, `getViews()`, `getTypes()`, `getColumns()`, `getIndexes()`, `getForeignKeys()`; Vorschau: `Blueprint::toSql()`, `Connection::pretend()`	Quellcode `vendor/laravel/framework` 13.39
Was hat Basalto, das hier wiederverwendbar ist?	Starter-Kit-Bereinigung mit Passkey- und Code-Anmeldung (TB 00, Scheibe 1), Tests auf drei bzw. vier Datenbanken, Diagramm-Generator und PDF-Bau der Doku, Lektionen zu Fortify und `laravel/passkeys` (Routen-Übernahme bei Minor-Update, globale Bindung von `{passkey}`) — eigenes Projekt, Übernahme erlaubt	`~/Development/basalto`, Vault [[Nguyen Development]]
Wie steht der Markt?	phpMyAdmin 5.2.3 nur bis PHP 8.3 freigegeben, phpPgAdmin ruht, Adminer 6.2.0 aktiv, Jaxon DbAdmin 0.35 als neuer PHP-Mitbewerber	Marktbild
Ist der Name frei?	`phpdbadmin.ch` frei, `phpdbadmin.com` belegt seit 30.08.2025; keine gleichnamige Software gefunden	Whois und Websuche 28.10.2026

3. Rückfragen

A. Rahmen und Governance

Nr.	Frage	Annahme (gilt, solange nichts dagegen spricht)	Antwort/Entscheid
R-01	**Wer trägt das Vorhaben?** Eigenständiges Produkt der Einzelfirma Nguyen Development (wie Basalto), privates Werkzeug (wie Fristo als Nebenprodukt) oder ein Modul von Basalto?	Eigenständiges Produkt von Nguyen Development, zuerst für den eigenen Gebrauch	✅ 28.10.2026: **wie Annahme** — Produkt von Nguyen Development (zusammen mit R-40: MIT)
R-02	**Rollen:** alle DSDM-Rollen in Personalunion? Gibt es einen Gegenleser (Business Ambassador)?	Personalunion; Hoang ist erster Nutzer; ein Laravel-Entwickler aus dem Umfeld als Gegenleser ab Alpha	
R-03	**Vorrang und Zeit** gegenüber Basalto (Beta Anfang 2027) und Fristo (Pilot Frühling 2027)	Basalto behält Vorrang bis zur Beta; phpdbadmin läuft in kleinen Timeboxes daneben, Stufen 0–1 zuerst	✅ 28.10.2026: **wie Annahme** — neben Basalto
R-04	**Name:** Arbeitstitel «phpdbadmin» behalten oder Namensfindung (wie bei Basalto)? Der Name ist nahe an phpMyAdmin und nennt Technik statt Nutzen	Arbeitstitel bis vor dem ersten öffentlichen Auftritt; Namensfindung als Prüfpunkt der Feasibility (F-10)	✅ 28.10.2026: **phpdbadmin ist der Name** (endgültig) — F-10 wird «Name absichern»: Markenrecherche, Domain `.ch`, eigenständiger Auftritt
R-05	**Arbeitsvertrag:** Gilt die Meldung der Nebenbeschäftigung wie bei Basalto auch hier?	Ja, Vorbedingung vor dem ersten öffentlichen Auftritt	
R-06	**Doku-Spiegel:** nur Repo und Vault?	Nur Repo und Vault-Projektnotiz	
R-07	**Repo:** GitHub `hoang-79/phpdbadmin`, privat?	Privat bis v1.0, dann nach Lizenzentscheid (R-40)	
R-08	**Branches und Auslieferung:** direkt auf `main`, gebündelte Pushes?	`main` lokal, Pushes gebündelt; Auslieferungsweg mit dem Betriebsmodell (R-11)	

B. Produkt und Positionierung

Nr.	Frage	Annahme	Antwort/Entscheid
R-10	**Zielgruppe zuerst:** Laravel-/PHP-Entwickler lokal, Betriebsteams auf Servern oder Fachleute mit Datenpflege?	Entwickler zuerst, Betriebsteams gleich danach (Rechte, Protokoll in v1.0); Fachleute profitieren von der Führung	
R-11	**Betriebsmodell:** (a) nur lokal (Herd, ein Benutzer) · (b) lokal und selbst gehostet auf einem Server (mehrere Konten) · (c) zusätzlich als SaaS	(b) — kein SaaS, weil dafür fremde Zugangsdaten und Netzwege auf unseren Servern lägen	✅ 28.10.2026: **wie Annahme** — lokal und selbst gehostet
R-12	**Mehrere Konten ab v1.0?**	Ja: Einladung, Rollen, Rechte je Verbindung (A-06, A-07)	

Nr.	Frage	Annahme	Antwort/Entscheid
R-13	**Was heisst «intuitiver» konkret** — gibt es Vorbilder, die dir gefallen (TablePlus, Beekeeper Studio, Tabellen-Editor von Supabase, Notion-Datenbanken)?	Arbeitsfläche wie TablePlus, Bearbeiten wie der Tabellen-Editor von Supabase, festgehalten in den Prinzipien P-01 bis P-11 und im Bedienkonzept	
R-14	**Weitere Datenbanken später** (SQL Server, Oracle, CockroachDB, ClickHouse, MongoDB, Redis)?	Nicht vor v1.2; die Dialekt-Schicht hält die Tür offen (SQL Server hat einen Laravel-Treiber)	
R-15	**Mindestversionen der Server**	MariaDB 10.6, MySQL 8.0, PostgreSQL 14, SQLite 3.35 — in der Feasibility prüfen	
R-16	**phpMyAdmin-Gewohnheiten** übernehmen (Reiter «Operationen», URL-Aufbau, Designer)?	Nein, eigene Struktur; Begriffe dort angelehnt, wo sie verbreitet sind	
R-17	Soll phpdbadmin seine **eigene App-Datenbank** als Verbindung zeigen dürfen?	Nein — sonst liessen sich Konten und Rechte am System vorbei ändern	

C. Funktionsumfang

Nr.	Frage	Annahme	Antwort/Entscheid
R-20	**Umfang v1.0** = Stufen 0–2 wie im Scope-Diagramm?	Ja	
R-21	Verwaltung der **Datenbank-Benutzer und Rechte** (GRANT) erst in v1.1?	Ja	
R-22	**Export-Formate** in v1.0: CSV, JSON, SQL; Excel später?	Ja	
R-23	**KI-Hilfe** für SQL: nur optional, eigener Schlüssel, nur das Schema wird gesendet, Stufe 4?	Ja	
R-24	**Laravel-Extras** (Migration aus Tabelle, Herd-Erkennung) erst in Stufe 4 oder früher als Unterscheidungsmerkmal?	Stufe 4	
R-25	Darf das **Datenraster waagrecht scrollen** (Ausnahme von «keine Scrollbalken»)?	Ja, nur dort: fixierte erste Spalte, Verlaufskante mit Chevron, Spaltenwahl	
R-26	**Rückgängig für Datenänderungen** (D-09) schon in v1.0?	Ja als Should, begrenzt auf eine Zeilenzahl je Änderung	

D. Technik und Betrieb

Nr.	Frage	Annahme	Antwort/Entscheid
R-30	**Umgebungen** ausser Dev: Testing, Demo, Produktion?	Bis v1.0 nur Dev; danach eine öffentliche Demo mit SQLite-Beispieldatenbanken (Ort offen: cyon wie Basalto oder Laravel Cloud)	
R-31	**App-Datenbank**	SQLite als Vorgabe, die anderen drei möglich (B-06)	
R-32	**Neue Pakete** für SQL-Editor (CodeMirror 6, MIT) und Raster-Virtualisierung (TanStack, MIT)	Probe in der Feasibility (`composer` / `npm` trocken), Entscheid per ADR, Freigabe durch Hoang	✅ 28.10.2026: probeweise installiert (Freigabe Hoang); Entscheid in ADR-005/006

Nr.	Frage	Annahme	Antwort/Entscheid
R-33	**MySQL in Herd** installieren (8.4, Port 3307) für die Testmatrix	Ja, in TB 00	✅ 28.10.2026: in der Feasibility installiert (Herd meldete einen Fehler, legte den Dienst aber an)
R-34	**Mail:** lokal Herd-Mail, sonst SMTP der Installation	Ja	
R-35	**Bezeichner** im Code und in der App-Datenbank	Englisch (wie Fristo und Basalto), Oberfläche Deutsch und Englisch	
R-36	**Zeitzone**	Speichern in UTC, Anzeige in Europe/Zurich bzw. der Zeitzone des Kontos	

E. Geschäft

Nr.	Frage	Annahme	Antwort/Entscheid
R-40	**Lizenz:** Open Source (MIT), Open Core oder kommerziell?	Open Source (MIT); Team-Funktionen später allenfalls als bezahlter Zusatz	✅ 28.10.2026: **wie Annahme** — MIT
R-41	**Vertrieb:** Git und `composer create-project`; Docker-Image?	Git und Composer ab v1.0, Docker-Image als Could	
R-42	**Bezug zu Basalto:** eigenständig oder später ein Basalto-Modul «Datenbank»?	Eigenständig; gemeinsamer Code nur über Pakete	

4. Entscheide aus der interaktiven Runde vom 28.10.2026

Nr.	Gefragt	Gewählt	Folge
R-01, R-40	Träger und Lizenz	Nguyen Development, MIT (Empfehlung)	Lizenzdatei MIT vor dem ersten öffentlichen Push; Abhängigkeiten MIT-verträglich (F-9); kein GPL-Code
R-11	Betriebsmodell	lokal und selbst gehostet (Empfehlung)	Konten, Einladung, Rechte je Verbindung ab v1.0; Hosts-Liste; Installationsweg für Server (B-05)
R-03	Vorrang	neben Basalto (Empfehlung)	kleine Timeboxes, Stufen 0–1 bis zur Alpha zuerst
R-04	Name	**phpdbadmin ist der Name** (statt Arbeitstitel mit späterer Namensfindung)	`phpdbadmin.ch` sichern (Hoang); Markenrecherche zur Nähe zu «phpMyAdmin» und eigenständiger Auftritt vor dem ersten öffentlichen Auftritt (F-10, Risiko R6)

Neue Rückfragen ergeben sich daraus keine; die übrigen Fragen laufen mit ihrer Annahme weiter.

5. Entscheide aus dem Review der Foundation vom 28.10.2026

Nr.	Gefragt	Gewählt	Folge
R-27	Schreibschutz einer Produktions-Verbindung auf Zeit aufheben (Wireframe E-05)	ja, als neue Anforderung (Empfehlung)	**V-11** in CR-001 Fassung 4, TB 04
R-28	Raster: endlos scrollen oder Seiten zu 50 (Wireframe zeigte Seiten)	endlos scrollen (Empfehlung)	ADR-006 bestätigt; Wireframe-Fusszeile auf Positionsanzeige umgestellt
R-29	PHP 8.5 als Mindestversion (ADR-011)	PHP 8.5 (Empfehlung)	ADR-011 angenommen; `composer.json` `^8.5` in TB 00

Nr.	Gefragt	Gewählt	Folge
R-30	Übrige Wireframe-Fragen F-2 bis F-6 (Anwenden-Knopf rot, Detail-Schublade breit, neue Zeilen oben, Name am Fremdschlüssel, Startseite)	wie empfohlen	in der Wireframe-README als entschieden geführt

Anhang C — Marktbild — Datenbank-Werkzeuge 2026

Stand: 28.10.2026 · v0.1 — aus einer Recherche am selben Tag. Jede Angabe nennt ihre Quelle und wie sie geprüft ist: **geöffnet** (Seite selbst gelesen) oder **Suche** (aus der Zusammenfassung einer Websuche, Seite nicht geöffnet — vor Verwendung in Texten für draussen nachprüfen).

1. Übersicht

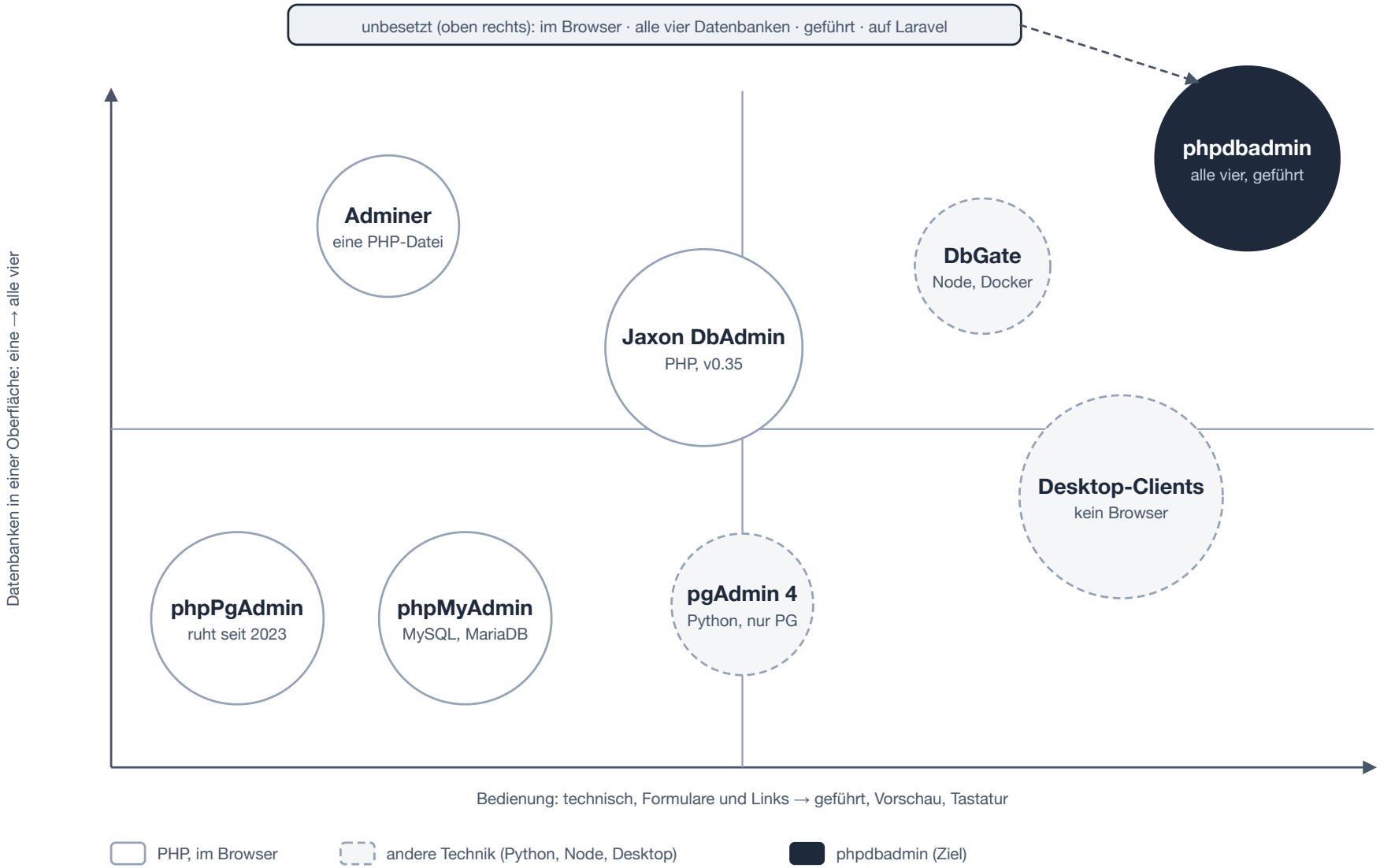
Werkzeug	Art	Datenbanken	Technik	Lizenz	Stand	Prüfung
phpMyAdmin	Web	MySQL, MariaDB	PHP; 5.2: PHP 7.2 bis unter 8.4	GPL-2.0	5.2.3 vom 08.10.2025; 6.0 «in Entwicklung» (täglicher Snapshot 27.10.2026, PHP 8.2 bis unter 8.4)	geöffnet [1]
phpPgAdmin	Web	PostgreSQL	PHP (ADOdb)	GPL-2.0	7.13.0; letzter Commit Dezember 2023 (Packagist); offene Meldung «RXSS» vom 11.03.2026; gepflegter Fork 7.14.8-mod	Suche [2]
Adminer	Web, eine Datei	MySQL, MariaDB, PostgreSQL, SQLite, MS SQL, Oracle, CockroachDB; weitere per Plugin	PHP	Apache-2.0 oder GPL-2.0 (zu bestätigen)	6.2.0 vom 23.10.2026; 6.0 brachte im August 2026 Anmeldung ohne Passwort	geöffnet [3]
pgAdmin 4	Web und Desktop	PostgreSQL	Python	PostgreSQL-Lizenz	9.18 vom 17.09.2026; monatliche Releases, je 4–8 behobene CVEs; KI-Funktionen seit 9.13	Suche [4]
DbGate	Web (Docker, npm) und Desktop	SQL und NoSQL, u. a. alle vier	Node.js	GPL-3.0 (Community) + Premium	7.3.0 vom 15.09.2026; Community laut Bericht nur Einzelbenutzer	Suche [5]
CloudBeaver	Web	sehr viele (DBeaver-Kern)	Java	Community + Enterprise	Version nicht geprüft	Suche [6]
Jaxon DbAdmin	Web, Paket für Laravel, Symfony, Slim	MySQL, MariaDB, PostgreSQL, SQLite	PHP; Funktionen aus Adminer, Bootstrap 5, CodeMirror 6	im Artikel nicht genannt	0.35.1, Artikel vom 08./13.10.2026; Audit-Log, Secret-Manager, Zielgruppe Entwicklerteams	geöffnet [7]

Werkzeug	Art	Datenbanken	Technik	Lizenz	Stand	Prüfung
Desktop-Clients	Desktop	je nach Produkt	TablePlus (nativ), Beekeeper Studio (Electron, Wechsel zu Tauri), DBeaver (Java), Sequel Ace (macOS), HeidiSQL (Windows)	bezahlt, Open Core oder GPL	Preise in Vergleichsartikeln widersprüchlich	Suche [6]

2. Was der Markt zeigt

phpdbadmin — Marktpositionierung

Datenbank-Werkzeuge 2026: wie viele Datenbanken in einer Oberfläche, gegen wie geführt die Bedienung ist (Marktbild § 3)



Nr.	Befund	Folge für phpdbadmin
B1	**Der PHP-Klassiker stockt.** phpMyAdmin 5.2 ist offiziell nicht für PHP 8.4 und neuer freigegeben, Version 6 hat keinen Termin.	Wer Herd oder einen neuen Server mit PHP 8.4/8.5 nutzt, hat keinen offiziell passenden phpMyAdmin.
B2	**phpPgAdmin ist faktisch verwaist.** Kein Commit seit 2023, Sicherheitsmeldung offen, neuere PostgreSQL-Versionen nur in Forks.	PostgreSQL im Browser heisst heute pgAdmin (schwer, Python) oder Adminer (knapp).
B3	**Adminer deckt die Breite ab** und ist aktiv, gewinnt aber durch Einfachheit (eine Datei), nicht durch Führung.	Die Breite allein ist kein Alleinstellungsmerkmal; die Bedienung ist es.
B4	**Moderne Oberflächen sind nicht PHP.** DbGate, CloudBeaver und die Desktop-Clients brauchen Node, Java, Docker oder eine Installation je Rechner.	Auf jedem PHP-Server und in Herd sofort lauffähig zu sein, ist ein Vorteil.
B5	**Jaxon DbAdmin zielt seit Oktober 2026 in dieselbe Lücke** — PHP, vier Datenbanken, Laravel-App, Audit-Log. Frühphase (0.35), Kern Adminer, Bootstrap.	Direkter Mitbewerber; beobachten. Unterscheidung über Bedienung (P-01 bis P-08), Anmeldung ohne Passwort und den Laravel-Stack von 2026.
B6	**KI-Hilfe für SQL wird Standard** (pgAdmin ab 9.13, DbGate 7.2.5, TablePlus-Plugin).	Als Option in Stufe 4, nur mit eigenem Schlüssel und nur mit dem Schema (R-23).
B7	**Sicherheit ist Dauerthema**: pgAdmin mit mehreren CVEs je Release, Adminer CVE-2026-16434 (bis 5.5.0), phpPgAdmin mit offener XSS-Meldung.	Sicherheit ab Werk ist Verkaufsargument und Pflicht (N-02).
B8	**Passwortlos kommt an**: Adminer 6.0 führt Anmeldung ohne Passwort ein.	Passkey und Code sind kein Alleinstellungsmerkmal mehr, aber Mindeststandard.

3. Positionierung

Option	Kern der Aussage	Bewertung
A — «Ein Werkzeug für alle vier, das führt»	phpMyAdmin und phpPgAdmin in einem, mit moderner, geführter Bedienung	trifft B1–B4; Kern des Vorschlags
B — «Das Datenbank-Werkzeug für Laravel-Entwickler»	Herd-Erkennung, Laravel-Migration aus Tabelle, Konventionen (`deleted_at`, `migrations`)	starkes Unterscheidungsmerkmal gegen Jaxon und Adminer; als Zusatz in Stufe 4
C — «Sicher im Team»	Konten mit Passkey, Rechte je Verbindung, Protokoll, Schreibschutz der Produktion	nötig gegen B7; Teil von A

Vorschlag: A mit C im Kern, B als Unterscheidungsmerkmal, sobald v1.0 steht. Leitsatz (Entwurf): *«Alle vier Datenbanken. Eine Oberfläche. Nichts passiert ungesehen.»*

4. Geschäftsmodelle im Markt

Modell	Beispiele	Erlös
Frei (Open Source)	phpMyAdmin, Adminer, pgAdmin, phpPgAdmin	Spenden, Sponsoren
Open Core	DbGate (Premium, Team Premium), Beekeeper Studio, CloudBeaver	Abo je Benutzer
Bezahlt	TablePlus, DataGrip	Lizenz oder Abo

Im Browser-Segment ist kostenlos die Regel. Ein Erlös wäre am ehesten mit Team-Funktionen möglich (Option B in der Ideensammlung § 9).

5. Risiken aus dem Markt

Nr.	Risiko	Gegenmittel
RM-1	Jaxon DbAdmin erreicht eine reife Version, bevor phpdbadmin v1.0 hat	früh nutzbare Stufen (Alpha lesend), Unterscheidung über Bedienung statt Funktionsliste
RM-2	phpMyAdmin 6 erscheint mit neuer Oberfläche und PHP-8.4-Freigabe	bleibt auf MySQL/MariaDB beschränkt; die Breite über vier Datenbanken bleibt
RM-3	Kostenloser Markt — kein Erlös	entschieden: Open Source unter MIT, Ziel ist der eigene Gebrauch und das Schaufenster (R-40)
RM-4	Name zu nahe an phpMyAdmin	Name bewusst gewählt (R-04); Markenrecherche und eigenständiger Auftritt vor dem ersten öffentlichen Auftritt (F-10)

6. Quellen (abgerufen 28.10.2026)

1. phpMyAdmin, Downloads — https://www.phpmyadmin.net/downloads/ (geöffnet)
2. phpPgAdmin: GitHub-Issues https://github.com/phppgadmin/phppgadmin/issues, Packagist https://packagist.org/packages/phppgadmin/phppgadmin, Fork https://github.com/ReimuHakurei/phppgadmin/releases (Suche)
3. Adminer — https://www.adminer.org/ (geöffnet)
4. pgAdmin, News — https://www.pgadmin.org/news/, https://www.postgresql.org/about/news/pgadmin-4-v917-released-3356/ (Suche)
5. DbGate, News — https://www.dbgate.io/news/; GIGAZINE 15.08.2026 https://gigazine.net/gsc_news/en/20260815-dbgate/ (Suche)
6. Vergleichsartikel 2026: https://www.bytebase.com/blog/top-dbeaver-alternative/, https://queryglow.com/blog/dbeaver-sql-client, http://aidev.fit/en/tools/best-database-gui-tools.html (Suche; Preise widersprüchlich)
7. «Jaxon DbAdmin: PHP finally gets a next-gen database manager», dev.to — https://dev.to/tiaya/jaxon-dbadmin-php-finally-gets-a-next-gen-database-manager-244h (geöffnet)
8. Endoflife phpMyAdmin — https://endoflife.date/phpmyadmin (Suche)

Anhang D — Funktionsraster — phpMyAdmin, phpPgAdmin, Adminer → phpdbadmin

Stand: 13.11.2026 · v0.2 (§ 10 Feinabgleich der Operationen). 28.10.2026 · v0.1. Grundlage: Controller und Seiten der lokalen Kopien von phpMyAdmin 5.2.2 und phpPgAdmin 7.13.0 (nur Dateinamen und Bereiche gelesen, kein Code übernommen), die Funktionsliste auf adminer.org (6.2.0) und der Artikel zu Jaxon DbAdmin (Marktbild § 6).

Zeichen: ✓ vorhanden · ◐ teilweise oder umständlich · — fehlt. **Vorschlag: v1.0** (Kern, Stufe 0–2) · **v1.1** (Verwalten, Stufe 3) · **v1.2** (Komfort und Team, Stufe 4) · **Nein**. Die Spalte «Anforderung» verweist auf CR-001 § 4.

1. Anmeldung und Zugang

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Anmeldung mit Datenbank-Benutzer und -Passwort	✓	✓	✓	**Nein** — eigene Konten	A-01
Eigene Konten der Anwendung	—	—	—	**v1.0**	A-06
Anmeldung ohne Passwort (Code per Mail)	—	—	◐ (ab 6.0)	**v1.0**	A-02
Passkey	—	—	—	**v1.0**	A-03
Zwei-Faktor (TOTP)	✓	—	◐ (Plugin)	**Nein** — Passkey ersetzt es	A-01
Rechte je Verbindung (lesen, schreiben, verwalten)	◐ (Benutzergruppen, Menüs ausblenden)	—	—	**v1.0**	A-07
Protokoll der Anmeldungen	—	—	—	**v1.0**	A-09

2. Verbindungen

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Mehrere Server	✓ (Konfigurationsdatei)	✓ (Konfigurationsdatei)	✓ (Eingabe beim Anmelden)	**v1.0** — in der Oberfläche anlegen	V-01
Verbindung testen mit verständlichem Fehler	◐	◐	◐	**v1.0**	V-02
Gespeicherte Zugangsdaten verschlüsselt	◐ (Cookie verschlüsselt)	—	◐ (Sitzung)	**v1.0**	V-03
Umgebung (lokal, Test, Produktion) mit Farbe, Schreibschutz	—	—	—	**v1.0**	V-04

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Erlaubte Hosts (kein Sprungbrett ins Netz)	◐ (`AllowArbitraryServer` aus)	◐	◐ (Plugin)	**v1.0**	V-05
SQLite-Dateien	—	—	✓	**v1.0**	V-01
SSH-Tunnel	—	—	—	**v1.2**	V-08
Herd-Dienste und `.env` erkennen	—	—	—	**v1.2**	V-07

3. Bedienung

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Navigationsbaum	✓	✓ (Baum, Frames)	◐ (Liste)	**v1.0**	O-01
Befehlspalette, Schnellsuche über Objekte	◐ (Filter im Baum)	—	—	**v1.0**	O-02
Reiter für mehrere offene Objekte	—	—	—	**v1.0**	O-03
Gleiche Bedienung über alle Datenbanken	—	—	✓	**v1.0**	O-04
Dunkelmodus	◐ (Themes)	—	◐ (Designs)	**v1.0**	O-06
Mehrsprachig	✓	✓	✓	**v1.0** DE/EN	O-06, N-06
Mobil lesbar	◐	—	◐	**v1.0** lesen	O-07

4. Daten

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Blättern, sortieren	✓	✓	✓	**v1.0**	D-01
Filtern geführt / freies WHERE	✓ (Suchen, Query by Example, Zoom-Suche)	◐ (Suchen)	✓	**v1.0**	D-02
Fremdschlüssel folgen	✓	—	✓	**v1.0**	D-03
Bearbeiten direkt in der Zelle	◐ (Doppelklick, je Zelle sofort gespeichert)	—	—	**v1.0** mit Sammeln und Vorschau	D-04
SQL-Vorschau vor dem Schreiben	◐ (nur «SQL anzeigen» nachher)	—	—	**v1.0**	D-04, P-02
Rückgängig für Datenänderungen	—	—	—	**v1.0** (begrenzt)	D-09
JSON, Binärdaten, Datum typgerecht	◐ (Transformationen)	◐	◐	**v1.0**	D-05
Geschätzte Zeilenzahl für grosse Tabellen	✓ (InnoDB-Schätzung)	—	✓	**v1.0**	D-06
Suchen und Ersetzen	✓	—	—	**v1.1**	D-08
Suche über alle Tabellen	✓	✓ (Finden)	✓	**v1.1**	D-08
Diagramm aus Abfrageergebnis	✓	—	—	**Nein** (BI)	—

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
GIS-Darstellung	✓	—	—	**Nein**	—

5. Struktur

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Spalten, Indizes, Fremdschlüssel ansehen	✓	✓	✓	**v1.0**	S-01
DDL anzeigen	✓	◐	✓	**v1.0**	S-01
Tabelle anlegen und ändern	✓	✓	✓	**v1.0** mit Vorschau	S-02, S-03
Umbenennen, kopieren, leeren, löschen	✓ (Operationen)	✓	✓	**v1.0** mit Namenseingabe	S-04
Datenbank oder Schema anlegen	✓	✓ (Schemas)	✓	**v1.0**	S-05
Views	✓	✓	✓	**v1.1**	S-06
Routinen, Trigger	✓	✓	✓	**v1.1**	S-06
Events (MySQL)	✓	—	✓	**v1.1**	S-06
Sequenzen, Typen, Domänen, Regeln (PostgreSQL)	—	✓	◐	**v1.1** (Sequenzen, Typen)	S-06
Partitionen	✓	—	◐	**v1.2**	—
Beziehungsdiagramm	✓ (Designer)	—	✓ (Schema)	**v1.1**	S-07
Datenwörterbuch, Normalisierung	✓	—	—	**Nein**	—
Änderungsverfolgung (Tracking)	✓	—	—	**Nein** — Protokoll deckt es	B-04
Schemavergleich	—	—	—	**v1.2**	S-08
Laravel-Migration aus Tabelle	—	—	—	**v1.2**	S-09

6. SQL

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Editor mit Hervorhebung	✓ (CodeMirror 5)	◐	◐	**v1.0**	Q-01
Autovervollständigung aus dem Schema	✓	—	—	**v1.0**	Q-01
Formatieren	✓	—	—	**v1.0**	Q-01
Mehrere Ergebnisse, Abbrechen	◐	—	◐	**v1.0**	Q-02
Zeitlimit, Zeilengrenze, nur lesen	◐	—	◐	**v1.0**	Q-03
Verlauf, gespeicherte Abfragen	✓ (Lesezeichen, Verlauf)	✓ (Verlauf)	◐ (Verlauf)	**v1.0**	Q-04
Gefährliche Anweisungen erkennen	◐ (Warnung bei DROP)	—	—	**v1.0**	Q-07
EXPLAIN	✓ (Tabelle)	✓	✓	**v1.1** als Baum	Q-06
Ergebnisse bearbeiten	✓	—	◐	**v1.1**	Q-05

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
KI-Hilfe	—	—	—	**v1.2** optional	Q-08

7. Export und Import

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Export SQL, CSV	✓	✓ (über pg_dump)	✓	**v1.0**	E-01
Export JSON	✓	—	◐ (Plugin)	**v1.0**	E-01
Export Excel, PDF, LaTeX u. a.	✓	—	—	**v1.1** Excel, sonst **Nein**	E-04
Import CSV mit Spaltenzuordnung	◐ (ohne Zuordnung)	✓	◐	**v1.0** mit Probelauf	E-02
Import SQL-Datei	✓	✓	✓	**v1.0**	E-02
Grosse Exporte im Hintergrund	—	—	—	**v1.0**	E-01
Kopieren als TSV, Markdown, INSERT	◐	—	—	**v1.0**	E-05
Sicherung und Wiederherstellung mit den Server-Werkzeugen	—	✓ (pg_dump)	—	**v1.1**	E-03

8. Server-Verwaltung

Funktion	phpMyAdmin	phpPgAdmin	Adminer	Vorschlag	Anforderung
Server-Übersicht, Grösse je Datenbank	✓	✓	◐	**v1.1**	W-01
Prozesse anzeigen und beenden	✓	✓	✓	**v1.1**	W-02
Variablen und Status	✓ (Monitor, Berater)	◐	✓	**v1.1** lesen	W-03
Benutzer und Rechte	✓	✓ (Rollen, Gruppen)	✓	**v1.1**	W-04
Wartung (ANALYZE, VACUUM, OPTIMIZE)	✓	✓	◐	**v1.1**	W-05
Replikation, Binärlog	✓	—	—	**Nein**	—
Tablespaces, Engines, Plugins	✓	✓ (Tablespaces)	—	**Nein** (nur anzeigen in W-01)	—

9. Zusammenfassung

Der Kern von phpdbadmin v1.0 deckt das ab, was alle drei Vorbilder gemeinsam haben (Blättern, Filtern, Struktur, SQL, Export, Import), und legt das darauf, was keines hat: **eigene Konten ohne Passwort, Rechte je Verbindung, Sammeln mit SQL-Vorschau, Rückgängig, Umgebungsfarbe mit**

Schreibschutz, Befehlspalette und gleiche Bedienung über vier Datenbanken. Die Server-Verwaltung, in der phpMyAdmin am breitesten ist, folgt bewusst in v1.1 — sie wird seltener gebraucht und ist je Datenbank am verschiedensten.

10. Feinabgleich: Operationen für Datenbank und Tabelle (13.11.2026)

Anlass: Hoang vermisste «db dump, db backup, db optimierung oder repair» und «für Tabellen Analyse, Optimierung etc.» wie in phpMyAdmin. Abgeglichen mit den Reitern «Operationen», «Export», «Import» und der Wartung je Tabelle in phpMyAdmin 5.2 und mit dem gebauten Stand von phpdbadmin (Routen, Menüs, Dialekte). Umgesetzt in Timebox 13 (13.11.2026): alle «neu» und «Menü» unten. **Befund:** Dump, Sicherung, Analysieren, Optimieren und Prüfen gab es schon — nur über das Menü des Verbindungs-Chips («Export und Import», «Server → Wartung»), nicht dort, wo man sie sucht.

Funktion	phpMyAdmin	phpdbadmin vorher	Entscheid	Anforderung
Export der Datenbank (SQL-Dump), einer Tabelle	✓ (Reiter Export)	✓, nur über Verbindungs-Chip bzw. Knopf im Raster	**Menü Datenbank / Weitere Aktionen**, vorgewählt (ganze Datenbank als SQL)	O-10, E-01
Import in Datenbank oder Tabelle	✓ (Reiter Import)	✓, nur über Verbindungs-Chip	**Menü**, vorgewählt	O-10, E-02
Sichern und Wiederherstellen mit `mysqldump` / `pg_dump`	— (nur Export)	✓, nur über «Export und Import»	**Menü Datenbank**	O-10, E-03
Analysieren, Prüfen, Optimieren je Tabelle und für alle	✓ (Operationen, Mehrfachauswahl)	✓ unter «Server → Wartung»	**Menü**, Art und Tabelle vorgewählt	O-10, W-05
Reparieren (`REPAIR TABLE`)	✓	—	**neu** (MySQL/MariaDB; InnoDB kann es nicht — als Befund)	W-06
Prüfsumme (`CHECKSUM TABLE`)	✓	—	**neu**	W-06
Indizes neu aufbauen (`REINDEX`)	—	—	**neu** (PostgreSQL, SQLite)	W-06
`FLUSH TABLE`, Defragmentieren (`ALTER … ENGINE`)	✓	—	**Nein** — Optimieren deckt es ab	—
Tabellenoptionen: Engine, Zeichensatz/Sortierung, `AUTO_INCREMENT`, Kommentar	✓ (Operationen)	◐ (nur Kommentar)	**neu**	S-10
Tabelle in andere Datenbank kopieren oder verschieben	✓	◐ (Kopie nur in derselben Datenbank)	**neu** (gleicher Server)	S-11
Zeichensatz/Sortierung der Datenbank ändern	✓	—	**neu**	S-12
Datenbank umbenennen oder kopieren	✓ (Tabelle für Tabelle, mit Löschen der alten)	—	**später** — MySQL kennt kein `RENAME DATABASE`; Kopie über Sichern und Wiederherstellen	—

Funktion	phpMyAdmin	phpdbadmin vorher	Entscheid	Anforderung
Server-Status (Zähler, Monitor, Berater), Engines, Zeichensätze, Plugins	✓	◐ (Übersicht, Variablen)	**später** (W-03 lesen erweitern)	W-03
Partitionen verwalten	✓	—	**später** (Plan v1.2)	—
Query by Example, visueller Abfrage-Baukasten	✓	◐ (Filter im Raster)	**Nein** — Filter, SQL-Editor und KI-Hilfe decken es	—

Anhang E — Ideen-Backlog — phpdbadmin

Stand: 28.10.2026 · erste Sammlung. Alle Ideen mit Nutzen, Herkunft und MoSCoW-Vorschlag. Was in CR-001 § 4 als Anforderung steht, trägt deren Nummer; die übrigen sind Kandidaten für spätere Change Requests. Herkunft: **H** Hoangs Auftrag · **M** Marktbild · **F** Funktionsraster · **L** Lektionen aus Basalto und Fristo · **C** Vorschlag Claude.

1. Anmeldung und Konten

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Anmeldung nur mit Passkey oder Code per Mail	kein Passwort, das man verlieren oder erraten kann	H	Must	A-01 bis A-03
Passkey nach der ersten Code-Anmeldung anbieten	schnellere Anmeldung im Alltag	C	Should	A-03
Einladung mit Ablauf statt Registrierung	keine fremden Konten	C	Must	A-06
Rechte je Verbindung (lesen, schreiben, verwalten)	Teams ohne geteilte DB-Passwörter	M	Should	A-07
Sitzungen sehen und beenden	verlorenes Gerät sperren	C	Could	A-08

2. Verbindungen

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Umgebungsfarbe und Schreibschutz für die Produktion	Verwechslung lokal/Produktion verhindern	C	Must	V-04
Rotes Favicon und Präfix im Seitentitel bei Produktion	Gefahr auch im Browser-Tab sichtbar	C	Should	V-04
«Passwort nicht speichern» je Verbindung	heikle Server ohne gespeicherte Zugangsdaten	C	Should	V-03
Erlaubte Hosts und Ordner	kein Sprungbrett ins interne Netz	L	Must	V-05
Herd-Dienste und `.env` der Herd-Sites erkennen	Verbindung mit einem Klick	C	Could	V-07
SSH-Tunnel	interne Server erreichen	F	Could	V-08
Verbindungsliste aus phpMyAdmin-Konfiguration übernehmen	Umstieg erleichtern	C	Won't	—

3. Bedienung

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Befehlspalette ⌘K	alles mit der Tastatur erreichen	C	Must	O-02
Reiter, die das Neuladen überleben	Arbeit nicht verlieren	C	Should	O-03
Zeilen-Detail als Seitenschublade mit allen Feldern und Beziehungen	lange Werte und Bezüge auf einen Blick	C	Should	D-05

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Karten statt Tabelle auf dem Telefon	lesbar ohne waagrechtes Scrollen	L	Could	O-07
Kürzel-Übersicht mit `?`	Tastatur lernen	C	Should	O-05

4. Daten

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Änderungen sammeln, SQL-Vorschau, einmal anwenden	nichts passiert ungesehen	H, C	Must	D-04
Rückgängig für angewendete Datenänderungen	Fehler in Sekunden beheben	C	Should	D-09
Betroffene Zeilen vor UPDATE und DELETE zählen	Ausmass vor dem Ausführen sehen	C	Must	Q-07
Fremdschlüssel vorwärts und rückwärts folgen	Zusammenhänge ohne SQL	F	Should	D-03
Spaltenstatistik (Füllgrad, NULL-Anteil, kleinster und grösster Wert, häufigste Werte)	Datenqualität messen statt Bildschirme ansehen	L	Could	—
Abfrage-Parameter (`:von` , `:bis`) mit Formular	gespeicherte Abfragen wiederverwenden	C	Could	Q-04
Vergleich zweier Abfrageergebnisse	«was hat sich geändert?»	C	Could	—
Testdaten erzeugen (Faker) für eine Tabelle	lokale Entwicklung	C	Could	—
Datenmaskierung für Bildschirmfreigaben (E-Mail, Telefon)	Vorführen ohne Datenschutzproblem	M	Could	—

5. Struktur und SQL

Idee	Nutzen	Herkunft	MoSCoW	Bezug
SQL-Vorschau für jede Strukturänderung	DDL verstehen und prüfen	H, C	Must	S-02, S-03
Gesundheitsprüfung: Fremdschlüssel ohne Index, Tabellen ohne Primärschlüssel, `TEXT` an der 64-KB-Grenze, `timestamp` ohne Vorgabe	Fallen aus Basalto früh sehen	L	Could	—
Laravel-Konventionen erkennen (`migrations` , `deleted_at` , `created_at`) — Filter «ohne gelöschte»	Laravel-Daten richtig lesen	C	Could	—
Laravel-Migration aus Tabelle	Bestehendes in Code überführen	C	Could	S-09
Schemavergleich mit Änderungsskript	Stände angleichen	F	Could	S-08
Beziehungsdiagramm	Datenmodell verstehen	F	Should	S-07
EXPLAIN als Baum mit Hinweisen	langsame Abfragen verstehen	F	Should	Q-06
SQL-Hilfe mit KI (eigener Schlüssel, nur Schema)	Abfragen schneller schreiben	M	Could	Q-08
«Erkläre diese Tabelle» mit KI	fremde Datenbanken verstehen	C	Won't (vorerst)	—

6. Export, Import, Betrieb

Idee	Nutzen	Herkunft	MoSCoW	Bezug
Export CSV, JSON, SQL im Hintergrund	grosse Tabellen ohne Zeitüberschreitung	F	Must	E-01
CSV-Import mit Spaltenzuordnung und Probelauf	Fehler vor dem Schreiben sehen	F, L	Must	E-02
Kopieren als TSV, Markdown, INSERT	schnell in Ticket oder Doku	C	Should	E-05
Sicherung mit `mariadb-dump`, `mysqldump`, `pg_dump`	Rückweg vor grossen Änderungen	F	Should	E-03
Protokoll aller schreibenden Aktionen	nachvollziehen, wer was geändert hat	M	Must	B-04
Docker-Image	Betrieb neben Datenbanken in Containern	M	Could	R-41
Öffentliche Demo mit SQLite-Beispieldatenbanken	ausprobieren ohne Installation	C	Should	R-30

Architektur-Hinweis

Fast jede Idee berührt die **Dialekt-Schicht**: Was eine Funktion je Datenbank anders tun muss, gehört dorthin und bekommt einen Vertragstest, der gegen alle vier Treiber läuft. Ideen, die nur auf einem Treiber sinnvoll sind (Events in MySQL, Sequenzen in PostgreSQL, `PRAGMA` in SQLite), erscheinen nur dort — mit derselben Bedienung.